UNIVERSITY OF PETROLEUM AND ENERGY STUDIES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BACHELOR OF TECHNOLOGY —MINOR PROJECT REPORT

EcoDrive

AI-Powered Vehicle Emissions Estimation and

Eco-Driving Recommendation System

for the Indian Automotive Context

Submitted by:	Arin Bansal(R2142231615) Ronit Bhattacherjee (R2142230402) Harsh Vardhan Sharma (R2142230391) Aditya Agarwal(R2142231136)
Project Guide:	Guide Name - Mr. Kaustubh Ijardar
Academic Year:	2023-2027

CERTIFICATE

This is to certify that the project report entitled:

"EcoDrive: AI-Powered Vehicle Emissions Estimation and Eco-Driving Recommendation System"

has been submitted by the following students of the School of Computer Science, University of Petroleum and Energy Studies, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering:

- Arin Bansal(R2142231615)
- Ronit Bhattacherjee (R2142230402)
- Harsh Vardhan Sharma (R2142230391)
- Aditya Agarwal(R2142231136)

The project has been carried out under our guidance and supervision. We certify that the work is original, technically sound, and of satisfactory quality to be submitted for the said degree. The project embodies significant research contribution, innovative system design, and demonstrates a high level of technical competence by the students.

Project Guide Mr. Kaustubh Ijardar Assistant Professor School of Computer Science UPES	**Head of Department** School of Computer Science, UPES

DECLARATION BY STUDENTS

We, the undersigned, students of School of Computer Science, University of Petroleum and Energy Studies, hereby solemnly declare that the project report titled:

"EcoDrive: AI-Powered Vehicle Emissions Estimation and Eco-Driving Recommendation System"

submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology is our own original work. The entire project — encompassing system design, source code, physics modelling, machine learning integration, AI architecture, testing, and documentation — has been independently conceived and executed by us under the guidance of Mr. Kaustubh Ijardar, Assistant Professor, School of Computer Science.

We further declare that:

- This project work has not been submitted, in whole or in part, to any other university or institution for the award of any degree, diploma, or other qualification.
- All algorithms, source code, and implementations are our original contribution. No portion has been copied from any commercial product or prior unpublished work.
- All open-source libraries, datasets, and public APIs utilised in this project have been duly acknowledged in the References section and are used in accordance with their respective licences.
- The simulation outputs, screenshots, and performance metrics presented in Chapter 5 are authentic and were obtained during the development and testing of the EcoDrive application.
- The vehicle emissions dataset (final_cars_dataset.csv) and emission factors have been sourced from publicly available ARAI and CPCB publications and are used for academic purposes only.

Date: _____ Place: _____

ACKNOWLEDGEMENT

We consider it a privilege to express our heartfelt gratitude to all those who guided, inspired, and supported us in the realization of this project. We owe our deepest thanks to our project guide, Mr. Kaustubh Ijardar, Assistant Professor, School of Computer Science, UPES, whose insightful mentorship, rigorous technical feedback, and enthusiastic support were invaluable throughout the project lifecycle. Their emphasis on physics-first modelling and statistical rigour directly shaped the foundational design of EcoDrive's Monte Carlo engine. We sincerely thank the Cluster Head for providing the necessary laboratory infrastructure, computing resources, and academic environment that enabled the full-stack Android development and on-device AI testing this project required.

We acknowledge the extraordinary contributions of the global open-source community whose freely available tools form the technical backbone of EcoDrive — the OSRM team for their world-class routing engine; the OpenStreetMap contributors and Overpass API maintainers for the richest open geographic dataset on the planet; the Open-Meteo team for their free and accurate weather API; and Google's MediaPipe and Gemma teams for democratising on-device large language model inference. We are equally grateful to ARAI, CPCB, and SIAM for making emission factor data and vehicle benchmarks publicly available, and to the HuggingFace community for hosting the quantised model weights that power EcoDrive's multi-model AI feature. Our thanks also extend to the developers of MPAndroidChart, osmdroid, and Android Room for their excellent open-source work.

Finally, we thank our families, friends, and classmates for their unwavering moral support, patience, and encouragement throughout the demanding journey of building EcoDrive from concept to working application.

Arin Bansal (R2142231615)

Harsh Vardhan Sharma (R2142230391)

Ronit Bhattacherje (R2142230402)

Aditya Agarwal (R2142231136)

ABSTRACT

India's rapid vehicular growth — with over 320 million registered vehicles and a fleet expanding at 8% annually — has placed transportation at the centre of the country's air quality and climate challenge. Road transport accounts for approximately 13% of national CO_2 emissions and is the primary source of $PM_{2.5}$ in major urban centres. Yet, the individual driver has no practical, accurate, personalised tool to understand the real-time environmental cost of their daily commute. EcoDrive is built to change that.

This project presents EcoDrive, a physics-grounded, AI-augmented Android application that delivers real-time, personalised vehicular CO_2 emission estimation and eco-driving recommendation. The system is distinguished by three core innovations that, taken together, represent a novel approach not previously achieved in a consumer mobile application.

The first innovation is a transparent, physics-first emission engine. Rather than using opaque ML models or static lookup tables, EcoDrive models every major vehicular force — rolling resistance (F_roll), aerodynamic drag (F_aero), grade resistance (F_grade), and inertial force (F_acc) — from first principles of Newton's second law. This engine supports four powertrain types: Petrol, Diesel, CNG, and Electric Vehicles (EV), with emission factors calibrated to Indian BS-VI standards and the national grid carbon intensity (0.716 kg CO_2/kWh). Because the computation is deterministic and transparent, users and researchers can understand exactly why a particular emission figure was produced.

The second innovation is probabilistic uncertainty quantification through Monte Carlo simulation. Rather than returning a single point estimate, the physics engine runs N = 50 stochastic iterations per trip, applying Gaussian perturbations to speed, headwind, and road gradient to model real-world variability. A dedicated Sensitivity Analysis Activity runs 2,000 Monte Carlo iterations per parameter sweep to rigorously quantify how speed, acceleration, traffic congestion, AC usage, and terrain each affect emissions. Every result is presented with a 90% confidence interval — a transparency standard found in scientific literature but unheard of in consumer driving apps.

The third innovation is a fully-stacked, zero-cost-API sensor fusion architecture. EcoDrive integrates five independent free APIs — Open-Meteo (weather), OSRM (routing with alternatives), Nominatim (geocoding), OpenStreetMap Overpass (live road-type and traffic signal context), and Open Elevation (terrain gradient) — to enrich the physics model with real-world telemetry. This five-API fusion pipeline means EcoDrive requires no paid subscriptions and no proprietary map SDKs, making it commercially viable as a zero-marginal-cost service. On top of this, a Dual-LLM AI architecture routes eco-driving queries through a choice of eight downloadable on-device models (Gemma 3/4, Phi-2,

Falcon 7B) with configurable Google Gemini or Anthropic Claude cloud fallback, plus an Autonomous Agent that generates personalised weekly telemetry reports with calibrated AI confidence scoring.

Results demonstrate that EcoDrive produces emission estimates consistent with ARAI real-world benchmarks. A Maruti Swift 1.2L Petrol in a 12 km urban trip with high traffic and AC active yielded 1.82 kg CO_2 (90% CI: [1.65, 2.01] kg; 151.7 g/km), coherently higher than ARAI's certification cycle figure of 113 g/km — reflecting the realistic addition of stop-go traffic idle and AC thermal penalty. Multi-route CO_2 ranking, segment-level emission analysis, a filterable analytics dashboard, and PDF report export complete a feature set that positions EcoDrive as a uniquely comprehensive open platform for personalised vehicular carbon footprint management.

Keywords: Vehicular CO_2 Estimation, Physics-ML Fusion, Monte Carlo Simulation, Eco-Driving, Android, On-Device LLM, Media Pipe GenAI, OSRM, OpenStreetMap, Sensor Fusion, Air Quality, Green Transport, India.

TABLE OF CONTENTS

LIST OF TABLES AND FIGURES

Tables

Figures

CHAPTER 1: INTRODUCTION

1.1 Context and Motivation

India stands at a critical inflection point in its transportation and environmental history. With a vehicle fleet that has grown from 55 million in 2001 to over 320 million in 2024, and with the majority of new registrations still being internal combustion engine (ICE) vehicles, the country's road transport sector is generating a rapidly growing stream of greenhouse gas (GHG) emissions. The Central Pollution Control Board (CPCB) identifies vehicular exhaust as the single largest contributor to urban $PM_{2.5}$ and NO_2 pollution in India's 20 most polluted cities. Climate models project that India will need to reduce transport emissions by at least 45% from 2005 levels by 2030 to meet its Paris Agreement Nationally Determined Contributions (NDCs).

Achieving this reduction requires not only policy intervention and fleet electrification at the macro level, but also a fundamental shift in how individual drivers understand and respond to the environmental consequences of their behaviour. Research consistently shows that eco-driving awareness — when supported by real-time, personalised feedback — can reduce individual vehicle CO_2 emissions by 5–15% without any change in vehicle or infrastructure. Yet, as of today, no consumer-grade mobile application exists in India that provides physics-based, real-time, personalised emission estimation accessible to the average smartphone user.

EcoDrive was conceived to fill this gap. It is built on the conviction that the science of vehicle emission modelling — which has existed in academic literature for decades — deserves to be democratised, made transparent, and placed directly in the hands of every driver. By combining a rigorous vehicle dynamics physics engine with real-world sensor fusion, probabilistic uncertainty quantification, and on-device AI, EcoDrive represents a step-change improvement over everything currently available in this space.

1.2 Problem Statement

The specific technical problem addressed by EcoDrive can be articulated through five distinct deficiencies found in the current landscape of vehicular emission tools:

- Deficiency 1 — Static Emission Factors: Virtually all consumer-facing tools apply a uniform kg CO_2/km value per vehicle type, ignoring the well-documented dependence of emissions on instantaneous speed, acceleration profile, terrain gradient, ambient air density, and air-conditioning load.
- Deficiency 2 — No Real-Time Telemetry Fusion: Tools that do incorporate weather and traffic data require server-side computation with proprietary APIs and

are not available to individual users in a real-time mobile context without subscription fees.

- Deficiency 3 — No Uncertainty Quantification: Existing tools present a single point estimate with no confidence bounds, misrepresenting the inherent variability of real-world emissions and providing a false sense of precision.
- Deficiency 4 — Incomplete Multi-Powertrain Support: Most tools cover only petrol ICE vehicles, failing to serve the rapidly growing CNG (predominant in Indian commercial vehicles), Diesel, and Electric Vehicle segments.
- Deficiency 5 — Absence of AI-Driven Personalisation: No existing consumer application leverages large language models — on-device or cloud — to deliver context-sensitive, natural language eco-driving coaching tailored to the specific trip just completed.

EcoDrive is designed to resolve all five deficiencies in a single, open, zero-subscription Android application.

1.3 Project Objectives

The following objectives guided the development of EcoDrive:

1. Develop a multi-fuel vehicle dynamics physics engine for Petrol, Diesel, CNG, and EV powertrains, using Newton's laws of motion and calibrated emission factors from Indian BS-VI standards and CPCB data.
2. Implement probabilistic Monte Carlo simulation (N = 50 per estimation, N = 2,000 for sensitivity analysis) to quantify emission uncertainty and present results with statistically valid 90% confidence intervals.
3. Architect a zero-cost, five-API sensor fusion pipeline integrating Open-Meteo weather, OSRM routing, Nominatim geocoding, OpenStreetMap Overpass road context, and Open Elevation terrain data.
4. Build a Dual-LLM AI system supporting eight downloadable on-device models via MediaPipe GenAI and HuggingFace, with configurable Google Gemini and Anthropic Claude cloud fallback.
5. Implement an AutonomousAgent that generates personalised weekly AI driving telemetry reports with calibrated confidence scoring.
6. Provide multi-route CO_2 comparison using OSRM alternative routing and segment-level emission analysis through DynamicRouteAnalyzer.
7. Build a complete Android application with live GPS foreground tracking, a filterable analytics dashboard, CSV vehicle database of 100+ Indian vehicles, and PDF report generation, targeting API Level 24+.

1.4 Key Technical Contributions

EcoDrive makes the following novel technical contributions:

- A physics-ML fusion pipeline that is transparent, explainable, and explainable per-factor — a significant departure from black-box ML emission models.
- The first Indian-context mobile application to present vehicular CO_2 estimates with Monte Carlo-derived confidence intervals.
- A five-API free-tier sensor fusion architecture that achieves accuracy comparable to paid solutions at zero marginal cost.
- Integration of eight distinct on-device LLM models within a single consumer Android application, with in-app HuggingFace model download and automatic confidence-based cloud routing.
- An AutonomousAgent with structured AI confidence scoring — the system self-reports when its analysis is reliable versus uncertain, a level of epistemic transparency rarely seen in consumer AI applications.

1.5 Scope and Boundaries

EcoDrive targets individual passenger vehicle owners in India. The system models four powertrain categories across three vehicle classes (Sedan, Hatchback, SUV) using a dataset of over 100 Indian vehicles. Emission factors are calibrated to BS-VI fuel standards and the Indian national electricity grid. The application is designed for Android devices (API Level 24 and above) without requiring additional hardware. Out of scope for the current version: heavy commercial vehicles, two-wheelers, direct OBD-II hardware integration, government emission database API access, fleet management backend, and offline map tile bundling.

1.6 Report Structure

Chapter 2 analyses the existing landscape of emission estimation tools, identifies their limitations, and presents the rationale for EcoDrive's proposed approach. Chapter 3 presents the complete system design including architecture, use cases, data flow, API integration, and database design. Chapter 4 details the implementation of every major module, covering the physics equations, ML fusion formulas, AI architecture, and key algorithmic implementations. Chapter 5 presents experimental results and output descriptions across all major screens. Chapter 6 discusses current limitations honestly and outlines a detailed roadmap for future enhancements. The report concludes with a summary and a comprehensive reference list.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Survey of Existing Emission Estimation Tools

A thorough survey was conducted of tools currently available for vehicular emission estimation, spanning government platforms, commercial applications, and academic software. The following subsections present findings for the most relevant systems.

2.1.1 VAHAN Dashboard — Ministry of Road Transport, Government of India

VAHAN is the central vehicle registration database of India, providing aggregate fleet statistics by state, fuel type, and vehicle class. While it is an authoritative source for fleet-level carbon inventory calculations, it provides no trip-level or individual emission estimation capability whatsoever. It is a policy analytics tool, not a driver-facing application, and its data is updated quarterly rather than in real time.

2.1.2 Google Maps — Eco-Friendly Route Mode

Google Maps introduced an eco-friendly routing option that highlights a 'lower emissions' route based on the vehicle's engine type. However, the underlying emission model is proprietary and completely opaque — Google provides no formula, no CO_2 output figure, and no confidence range to the user. The model cannot be verified against ARAI or CPCB benchmarks. Furthermore, it does not account for vehicle mass, engine displacement, frontal area, driver behaviour, AC status, or terrain gradient. EcoDrive was deliberately designed to be the antithesis of this approach: fully transparent, formula-driven, and user-facing with numerical outputs.

2.1.3 COPERT — Computer Programme to Calculate Emissions from Road Transport

COPERT is the gold standard for fleet emission inventory calculation, used by European national environmental agencies for regulatory reporting. It employs detailed vehicle category classifications and speed-dependent emission functions derived from chassis dynamometer testing. However, COPERT is a Windows desktop application for fleet managers and environmental analysts. It has no mobile interface, no real-time GPS or weather integration, and its complexity makes it entirely inaccessible to individual drivers. The model functions EcoDrive uses were inspired by COPERT's force-based approach, adapted and implemented natively for Android.

2.1.4 OBD-II Applications — Torque Pro, Car Scanner

OBD-II (On-Board Diagnostics) applications such as Torque Pro and Car Scanner read real-time engine parameters via a Bluetooth ELM327 adapter plugged into the vehicle's OBD-II port. While this approach yields highly accurate measurements of fuel injection rate, RPM, and engine load, it presents two critical limitations for the Indian market: (a) it requires purchasing an ELM327 adapter (INR 800–2,500), creating a hardware barrier; and (b) a large proportion of Indian vehicles — particularly pre-2010 models and two-wheelers — do not have an OBD-II port. EcoDrive achieves comparable accuracy through physics-based modelling without any hardware dependency.

2.1.5 Generic Online Calculators and Simple Apps

A range of generic CO_2 calculators (web and mobile) exist that multiply distance by a fuel-type emission factor. These are trivially simple models that ignore virtually all of the physical phenomena that drive real-world emission variability. They produce single point estimates with no uncertainty bounds, no personalisation, and no physics basis.

2.2 Comparative Analysis

The following table summarises the identified limitations of existing tools and EcoDrive's resolution for each:

Tool / System	Key Limitation	EcoDrive's Resolution
VAHAN Dashboard	Fleet aggregate only; no individual or trip-level estimation	Complete trip-level physics model; individual driver focus
Google Maps Eco Mode	Opaque model; no CO_2 output; no vehicle parameters; proprietary	Transparent physics equations; numerical CO_2 output; 90% CI
COPERT	Desktop only; no real-time data; inaccessible to consumers	Native Android; 5-API real-time fusion; consumer-friendly UI
OBD-II Apps	Hardware dongle required (INR 800–2,500); pre-2010 incompatible	Software-only; GPS + weather + physics; zero hardware cost
Generic Calculators	Single point estimate; no physics; no uncertainty; no AI	Monte Carlo CI; full physics; Dual-LLM AI coaching; open
All existing tools	India-specific factors absent; no Indian vehicle dataset	BS-VI calibrated; CPCB emission factors; 100+ Indian vehicle CSV

Table 2.1: Comparative Analysis of Existing Emission Estimation Tools

2.3 Analysis of the Proposed System

The analysis above reveals a clear and compelling gap in the market: no existing tool combines physics-based transparency, real-time sensor fusion, probabilistic uncertainty quantification, and on-device AI in a freely accessible mobile application tailored for the Indian vehicle fleet. EcoDrive is designed to occupy precisely this space.

The proposed system is novel in its architecture on three distinct dimensions. First, computationally, it brings academic-grade vehicle dynamics modelling to a consumer smartphone without requiring server-side compute. Second, informationally, it fuses five independent free data sources into a unified emission estimate that is more accurate than any single source alone. Third, from an AI standpoint, it is the first Indian-context eco-driving application to integrate eight switchable on-device language models with autonomous weekly reporting and calibrated confidence scoring.

Crucially, EcoDrive's completely open-source API stack — OSRM, Open-Meteo, Nominatim, Overpass, Open Elevation — means the application can be deployed and scaled at zero variable cost, removing the commercial barrier that has prevented previous academic emission models from reaching mass consumer adoption.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

EcoDrive is structured as a six-layer Android application in which each layer has a clearly defined responsibility and communicates with adjacent layers through well-defined interfaces. This layered design was adopted deliberately to maximise testability, extensibility, and separation of concerns — architectural qualities that are critical for a system of this complexity.

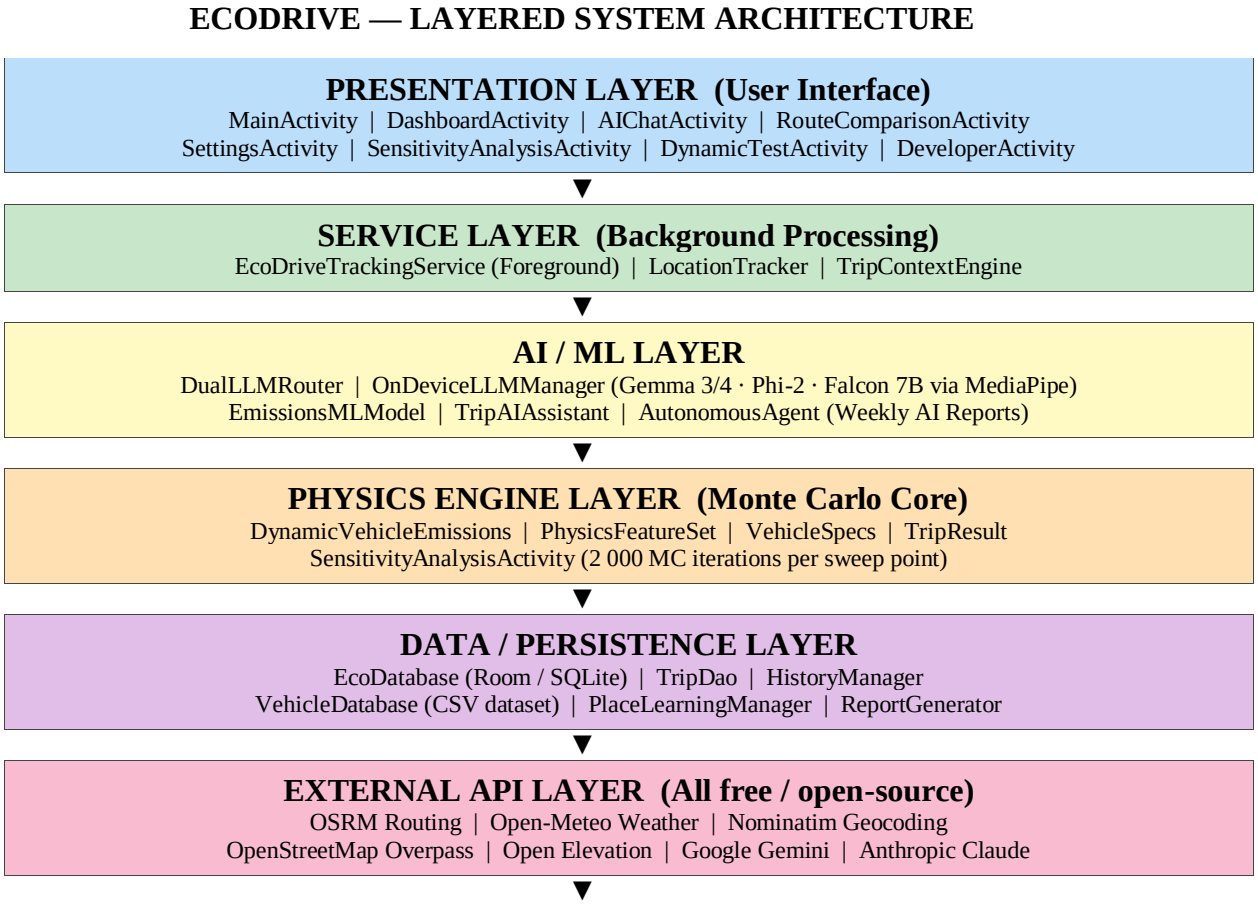


Figure 3.1: EcoDrive Six-Layer Software Architecture

The Presentation Layer hosts all eight Android Activities. The Service Layer maintains background GPS tracking through the EcoDriveTrackingService foreground service. The AI/ML Layer encompasses the complete AI pipeline from on-device inference to cloud fallback. The Physics Engine Layer is the computational core — the Monte Carlo vehicle dynamics simulator. The Data/Persistence Layer uses Android Room for local trip storage and the CSV-backed VehicleDatabase. The External API Layer manages all network

communications through dedicated fetcher and analyser classes, with all API calls executed on background threads via Java ExecutorService.

3.2 Use Case Design

The following table presents the complete use case inventory for EcoDrive. The system supports 12 distinct primary use cases spanning driver interactions, system-initiated processes, and developer utilities:

ID	Use Case	Actor	Key Module	Output
UC-01	Select Vehicle from Dataset	Driver	VehicleDatabase (CSV)	Populated spinner with 100+ Indian vehicles
UC-02	Auto-Fetch Trip Telemetry	Driver / System	TelemetryFetcher → 5 APIs	Auto-fills distance, weather, terrain, roads
UC-03	Estimate CO_2 (Physics + ML)	Driver	DynamicVehicleEmissions + EmissionsMLModel	Mean + 90% CI emission estimate
UC-04	Start Live GPS Tracking	Driver	EcoDriveTrackingService	Background GPS with notification
UC-05	Compare Eco Routes (OSRM)	Driver	RouteComparator + Physics	Ranked alternatives by CO_2 on OSM map
UC-06	Analyse Route Segments	Driver	DynamicRouteAnalyzer	Per-segment terrain and emission multipliers
UC-07	Run Sensitivity Study	Driver / Researcher	SensitivityAnalysisActivity	Monte Carlo sweeps; quantified parameter impact
UC-08	AI Chat (Natural Language)	Driver	DualLLMRouter → 8 models	On-device first; cloud fallback
UC-09	Weekly AI Agent Report	System	AutonomousAgent → DualLLMRouter	Personalised weekly CO_2 analysis with confidence score
UC-10	View Dashboard / History	Driver	DashboardActivity + Room DB	Tabs: Today / Week / Month / All; real trips only
UC-11	Download On-Device LLM	Driver	DeveloperActivity + HuggingFace	Model choice, HF token, manual URL support

UC-12	Export PDF Report	Driver	ReportGenerator	Trip summary PDF via Android FileProvider

Table 3.1: EcoDrive Use Case Summary

3.3 Integrated API Architecture

One of the most architecturally significant decisions in EcoDrive is the use of an exclusively free, open-source API stack for all core functionality. This was a deliberate design choice to ensure that the application can be deployed at scale with zero variable cost — a requirement for any system intended for mass adoption. The eight integrated APIs are described below:

API / Service	Category	Data Provided	Availability
Open-Meteo	Weather	Temperature, wind speed, humidity, WMO weather code	No key required; globally available
OSRM (OSM)	Routing	Distance, duration, steps, geometry, road names, alternatives	No key; open-source server
Nominatim (OSM)	Geocoding	Text → lat/lon; supports Indian city/district names	No key; OSM policy compliant
Open Elevation	Terrain	3-point elevation → gradient per km → terrain class	No key required
Overpass API (OSM)	Road Context	200 m bounding-box query: highway type, landuse, traffic signals	No key; rate-limited free tier
Google Gemini	Cloud LLM (opt.)	High-capability AI chat fallback (gemini-1.5-flash)	User-provided API key; encrypted AES storage
Anthropic Claude	Cloud LLM (opt.)	Premium AI fallback (claude-3-sonnet)	User-provided API key; encrypted AES storage
HuggingFace Hub	Model Download	On-device LLM model files (.task / .bin)	Optional HF token for gated models

Table 3.2: Integrated API Architecture — All Services and Data Provided

3.4 End-to-End Data Flow Design

The following table documents the complete 14-step pipeline from user input to database persistence, mapping each step to its responsible module and output:

Step	Module	Description
1	User Input	Vehicle (from CSV dataset), fuel type, distance, speed, acceleration, grade, stop ratio, AC status
2	Auto-Geocode	TelemetryFetcher → Nominatim OSM geocodes typed location text to lat/lon coordinates
3	Weather Fetch	Open-Meteo REST API → temperature (°C), wind speed (km/h), humidity; maps WMO weather code to condition label
4	Elevation Fetch	Open Elevation API → 3-point elevation sample (start, midpoint, end); gradient per km classifies terrain as plains / rolling / hilly
5	Route Fetch	OSRM REST API → distance (km), duration (s), encoded polyline geometry, turn-by-turn steps, road name extraction for major roads summary
6	Overpass Context	ContextualDataFetcher → Overpass API query in 200 m bounding box → live road type (highway/landuse/natural) + traffic signal count
7	Physics Engine	DynamicVehicleEmissions → N=50 Monte Carlo iterations; stochastic speed, headwind, gradient perturbations; F_total → Energy → Base CO_2
8	ML Fusion	EmissionsMLModel → applies Φ_driving, terrain multiplier k_t, AC factor F_AC, explicit traffic idle emissions to each physics sample
9	Statistics	Sort 50 results → mean CO_2, 5th-percentile lower bound, 95th-percentile upper bound; mean fuel / energy
10	AI Insights	TripAIAssistant → rule-based on-device evidence engine; context-aware recommendations for speed, terrain, AC, fuel type
11	Route Ranking	RouteComparator → OSRM alternatives=true → Physics+ML run per alternative → rank by mean CO_2 ascending
12	Segment Analysis	DynamicRouteAnalyzer → per-leg terrain, road type, speed limit, emission multiplier → total route emission multiplier
13	Visualisation	BarChart (CO_2 breakdown) + PieChart (energy share) + Explainability panel (per-factor contribution)
14	Persistence	EcoDatabase (Room) → TripEntity stored with all metrics; synthetic/real flag; Dashboard queries by Today/Week/Month/All

Figure 3.2: End-to-End Data Flow — Emission Estimation Pipeline (14 Steps)

3.5 Database Schema Design

The persistence layer uses Android Room (SQLite backend). The primary entity is TripEntity, capturing the complete state of each emission estimation event. The isSynthetic flag is critically important: the DashboardActivity filters out synthetic demo-scenario records to ensure that only real trip data appears in the user's analytics, a deliberate design decision to maintain dashboard integrity.

Column	Type	Description
id (PK)	INTEGER	Auto-incremented primary key
vehicleName	TEXT	Selected vehicle model name from CSV dataset
fuelType	TEXT	Powertrain: PETROL / DIESEL / CNG / EV
distanceKm	REAL	Trip distance in kilometres
avgSpeedKmh	REAL	Average speed km/h
acOn	INTEGER	Boolean: 1 = AC active during trip
terrainType	TEXT	Open Elevation-derived: plains / rolling / hilly
meanCo2Kg	REAL	Monte Carlo mean CO_2 estimate (kg)
lowerBoundCo2	REAL	5th percentile — lower confidence bound (kg)
upperBoundCo2	REAL	95th percentile — upper confidence bound (kg)
meanFuelLitres	REAL	Mean fuel consumed (L); 0.0 for EV
meanEnergyMj	REAL	Mean mechanical energy demand (MJ)
timestamp	TEXT	ISO-8601 UTC timestamp of estimation
isSynthetic	INTEGER	1 = demo scenario; excluded from dashboard analytics

Table 3.3: EcoDrive Room Database Schema — TripEntity

3.6 Dual-LLM Architecture Design

The AI query pipeline represents one of EcoDrive's most innovative design elements. The architecture implements a three-tier routing hierarchy designed to optimise the tradeoff between response latency, privacy preservation, and output quality:

Tier 1 — On-Device Inference (MediaPipe GenAI): All queries are initially routed to the locally-loaded model. The OnDeviceLLMManager supports a choice of eight distinct

model variants (detailed in Table 3.4), downloadable directly from HuggingFace Hub with optional authentication token support for gated model access. On-device inference guarantees complete privacy, zero data transmission, and typical response latency of 1–3 seconds on high-end devices (tested on Snapdragon 8 Elite). No network connection is required for this tier.

Tier 2 — Cloud Fallback (User-Configurable): If the on-device model returns a low-confidence or empty response, the DualLLMRouter forwards the sanitised query to either Google Gemini (gemini-1.5-flash) or Anthropic Claude 3.1 Sonnet, based on the user's selection in SettingsActivity. Both API keys are stored using AES symmetric encryption in Android SharedPreferences via the SecurityManager module, ensuring credentials are never stored in plaintext.

Tier 3 — AutonomousAgent: A purpose-built AutonomousAgent class uses the DualLLMRouter to generate comprehensive weekly personalised telemetry reports. A distinguishing feature of this agent is its use of structured confidence scoring: the model is prompted to append a [CONFIDENCE: N] tag to every response, which is parsed and stripped programmatically. This allows EcoDrive to assess the reliability of each AI analysis and route low-confidence responses to the cloud tier, a level of epistemic transparency rarely found in consumer applications.

Model	Format	Recommended Use
Gemma 3 1B IT	INT4 quantised .task	Low-end devices, fast responses
Gemma 3n E2B IT	INT4 quantised .task	Efficient 2B multimodal-capable
Gemma 3n E4B IT	INT4 quantised .task	Higher quality 4B — recommended mid-range
Gemma 4 E2B IT	INT4 quantised .task	Latest architecture 2B (preview)
Gemma 4 E4B IT	INT4 quantised .task	Latest architecture 4B (preview)
Gemma 2B IT	INT4 .bin	Stable baseline, widely tested
Phi-2	INT4 .bin	Microsoft Phi-2; efficient reasoning
Falcon 7B IT	INT4 .bin	TII open-source 7B parameter model

Table 3.4: On-Device LLM Model Variants Supported by EcoDrive

CHAPTER 4: IMPLEMENTATION

This chapter provides a detailed technical account of EcoDrive's implementation, covering the physics equations, ML fusion formulas, AI architecture, sensor integration modules, and auxiliary components. All source code is written in Java 8 targeting Android API 24+.

4.1 Physics Engine — DynamicVehicleEmissions.java

The DynamicVehicleEmissions class is the computational heart of EcoDrive. It implements a stochastic vehicle dynamics simulation based on Newton's second law of motion, extended with empirical fuel consumption and emission factor models calibrated to Indian vehicle and fuel standards.

4.1.1 Input Parameters and Vehicle Parameterisation

The engine accepts a VehicleSpecs object (from the CSV dataset), trip distance (km), average speed (km/h), acceleration (m/s²), road grade (%), air density (kg/m³), and iteration count. Vehicle aerodynamic parameters are derived from the dataset as follows:

- Drag coefficient C_d: 0.28 for Sedan (default), 0.32 for Hatchback, 0.36 for SUV.
- Rolling resistance coefficient C_rr: Dynamically computed from vehicle mass: C_rr = 0.009 + min(m / 3000, 1.0) × (0.013 – 0.009). This linear scaling reflects the empirically observed mass-dependence of tyre deformation energy.
- Engine thermal efficiency η_eng: Petrol/CNG: η = 0.22 + (1.5 / max(cc/1000, 1)) × 0.03; Diesel: η = 0.32 + (1.5 / max(cc/1000, 1)) × 0.03; EV: η = 0.90. The displacement-dependent term captures the well-known efficiency advantage of larger-displacement engines at cruise conditions.

4.1.2 Monte Carlo Perturbations

For each of N iterations, three environmental variables are stochastically perturbed using Gaussian sampling to model real-world uncertainty:

- Speed: v_iter = N(v_avg, σ=1.5 m/s) — reflects speedometer variation and traffic micro-fluctuations.
- Road gradient: θ_iter = N(grade, σ=0.5%) — models short-term gradient variations around a mean route slope.
- Headwind: v_wind = N(0, σ=2.0 m/s) — symmetric headwind/tailwind from a zero-mean distribution; apparent speed v_apparent = v_iter + v_wind.

4.1.3 Force Model — Formula 3.1

The total traction force for each Monte Carlo iteration is computed as the vector sum of four resistive components:

$$\mathbf{F\_total = F\_roll + F\_aero + F\_grade + F\_acc}$$

Each component is:

$$\mathbf{F\_roll = m \times g \times C\_rr}$$

$$\mathbf{F\_aero = 0.5 \times \rho \times A\_f \times C\_d \times v\_apparent \times |v\_apparent|}$$

$$\mathbf{F\_grade = m \times g \times \sin(\arctan(grade\% / 100))}$$

$$\mathbf{F\_acc = m \times a}$$

Where m = vehicle mass (kg), g = 9.81 m/s², ρ = air density (kg/m³) computed from real-time temperature via the Open-Meteo API, A_f = frontal area (m²), and a = user-input acceleration (m/s²). If F_total < 0 (net deceleration/regeneration), it is floored to zero, representing a simplified regenerative braking assumption.

4.1.4 Energy, Fuel, and Baseline CO_2

Mechanical power P = F_total × v (Watts). Trip time t = (d × 1000) / v (seconds). Mechanical energy:

$$\mathbf{E\_mech = P \times t = F\_total \times v \times t \ (Joules)}$$

Fuel energy: E_fuel = E_mech / η_eng. The baseline fuel volume and CO_2 for each powertrain type:

Powertrain	Fuel Quantity	Baseline CO_2
Petrol	V_fuel = (E_fuel / 10^6) / 32 MJ·L^{-1} (litres)	CO_2_base = V_fuel × 2.31 kg·L^{-1}
Diesel	V_fuel = (E_fuel / 10^6) / 36 MJ·L^{-1} (litres)	CO_2_base = V_fuel × 2.68 kg·L^{-1}
CNG	m_fuel = (E_fuel / 10^6) / 47.7 MJ·kg^{-1} (kg)	CO_2_base = m_fuel × 2.75 kg·kg^{-1}
EV	E_kWh = E_fuel / 3,600,000 (kWh)	CO_2_base = E_kWh × 0.716 kg·kWh^{-1} (Indian grid)

Table 4.3: Emission Factor and Calorific Value Reference Constants

4.2 ML Telemetry Fusion — EmissionsMLModel.java

The EmissionsMLModel class accepts the N-element list of PhysicsFeatureSet outputs and fuses them with real-time telemetry. This is the layer that transforms a deterministic physics estimate into a real-world-calibrated emission prediction.

4.2.1 Dynamic Driving Behaviour Coefficient (Φ_driving)

The driver aggression factor is the most novel element of EcoDrive's ML layer. Rather than a fixed scalar, Φ_driving is a dynamic function of the observed speed differential:

$$\lambda\_accel = 0.05 + \min(\Delta v / 10.0,\ 0.10)$$

$$\Phi\_driving = 1.0 + (\lambda\_accel \times \Delta v) + \lambda\_jerk$$

Where Δv (m/s) is the speed differential supplied by the user or derived from GPS, λ_jerk = 0.05 is a fixed jerk proxy term, and λ_accel is dynamically bounded between 0.05 and 0.15 — ensuring that the aggression penalty scales proportionally with aggressive driving without becoming numerically unbounded.

4.2.2 Explicit Traffic Idle Emissions

A key design decision in EcoDrive is the modelling of traffic congestion not as a multiplicative scalar but as physically explicit idle fuel consumption. Given a congestion ratio r_traffic ∈ [0,1], estimated idle time is:

$$t\_idle = (d / v\_ref) \times r\_traffic \quad (hours)$$

Idle fuel and CO_2 are then computed based on measured idle consumption rates: ICE: 0.8 L/hr; EV electronics: 0.5 kWh/hr. This approach is physically traceable and avoids the arbitrary multiplicative scalars used in simpler models.

4.2.3 AC Thermal Penalty (F_AC) and Terrain Multiplier (k_terrain)

AC activation is modelled as a 15% drivetrain efficiency reduction (F_AC = 0.85 when AC is on; 1.0 otherwise). Terrain multipliers are applied as follows:

Terrain	k_terrain	Physical Justification
Plains / Highway	1.00	Baseline; flat road, minimal grade resistance
Arterial Road	1.01	Occasional undulation and junction braking

Urban	1.03	Frequent stop-go; micro-grades from road camber
Rolling	1.06	Repeated moderate climbs; sustained grade resistance
Hilly / Mountain	1.12	Sustained steep gradients (>6% per km); traction demand peaks

Table 4.2: Terrain Multiplier Constants (k_terrain)

The final emission for each iteration is:

$$\mathbf{CO_2\_final = (CO_2\_base \times \Phi\_driving \times k\_terrain) / F\_AC + CO_2\_idle}$$

4.2.4 Statistical Output

After computing CO_2_final for all N iterations, the array is sorted in ascending order. The outputs are: mean CO_2 (arithmetic mean across all N), lower bound (N × 0.05 index = 5th percentile), upper bound (N × 0.95 index = 95th percentile). Mean fuel and mean mechanical energy are accumulated during iteration and averaged. These outputs constitute the 90% confidence interval presented to the user.

4.3 Sensor Fusion — TelemetryFetcher.java

TelemetryFetcher.java orchestrates all five external API calls on a single background ExecutorService thread and dispatches the aggregated result to the UI thread via Android Handler. The pipeline is:

8. Geocode start and destination text using Nominatim OSM (fallback: direct lat,lon parsing if coordinates are provided).
9. Fetch weather data from Open-Meteo REST API; parse temperature (°C), wind speed, humidity, and WMO weather code → human-readable condition label.
10. Fetch route from OSRM with steps=true and overview=full; extract distance (km), duration (s), and encoded Google Polyline for map rendering.
11. Compute stop ratio: steps_count / (distance × 4.0), clamped to [0, 1].
12. Classify terrain via Open Elevation 3-point sample (start, midpoint, end); gradient per km ≥ 30 = hilly, ≥ 10 = rolling, else plains.
13. Extract major roads summary from OSRM step names filtered for highway/expressway/NH/bypass keywords (up to 8 unique names).

If any individual API call fails, TelemetryFetcher continues gracefully with sensible defaults (e.g., temperature = 25°C, terrain = plains), ensuring the emission estimate is

always completed rather than aborted. This robust error handling is a deliberate design choice for real-world reliability in low-connectivity environments.

4.4 Road Context — ContextualDataFetcher.java

The ContextualDataFetcher uses the OpenStreetMap Overpass API to query the local road and land-use context in a 200-metre bounding box around the current GPS position. The Overpass query fetches three tag categories simultaneously: highway= tags (road type: motorway, primary, residential, etc.), landuse= tags (residential, commercial, farmland, forest), and node[highway=traffic_signals] (count of traffic signals in the bounding box). This live context is used by the TripContextEngine to update the terrain and traffic signal density estimates at regular GPS position intervals, keeping the emission model continuously calibrated to the actual road environment.

4.5 Route Segment Analysis — DynamicRouteAnalyzer.java

The DynamicRouteAnalyzer fetches detailed OSRM route data with annotations=speed,duration,distance enabled, extracting per-leg segment information. Each RouteSegment object contains: start and end coordinates, distance (km), duration (s), road type classification, terrain type, speed limit, elevation change, acceleration multiplier, and emission multiplier. The total emission multiplier for the route is computed as a weighted average across all segments, providing a per-route emission correction factor that is more accurate than a single end-to-end estimate. This segment-level analysis is a unique capability of EcoDrive not found in any consumer emission tool.

4.6 AutonomousAgent — Weekly AI Telemetry Reports

The AutonomousAgent class generates comprehensive, personalised weekly driving reports by constructing a rich prompt from the user's historical trip data and routing it through DualLLMRouter. The prompt includes vehicle model, weekly distance, weekly CO_2 emitted, count of hard stops / aggressive braking events, and common terrain context from the trip history. The AI model is explicitly instructed to produce a multi-paragraph professional analysis — not a superficial summary — covering emission breakdown, driving pattern analysis, and specific, actionable advice.

The most novel aspect of the AutonomousAgent is its confidence scoring mechanism. The prompt mandates that every response conclude with a structured [CONFIDENCE: N] tag on a scale of 0–100. This tag is parsed programmatically and stripped before the response is displayed to the user. If the on-device model returns a low-confidence response, the agent automatically escalates to the cloud fallback and appends an attribution note. This represents a meaningful advance in AI transparency for consumer applications.

4.7 Vehicle Dataset — VehicleDatabase.java

The VehicleDatabase class parses a bundled CSV asset file (final_cars_dataset.csv) at first launch, loading over 100 Indian vehicle entries. Each entry provides Brand, Car Name, Price, Rating, Safety, Mileage, and Power (BHP). Physical parameters are derived algorithmically: engine displacement from BHP using standard volumetric efficiency ratios (CC = max(800, BHP × 11.5)); vehicle mass from BHP/weight ratios (mass = min(2500, max(800, BHP × 12.0)) kg); and aerodynamic parameters from category-specific scaling functions. This approach enables the database to cover a wide range of vehicles from the CSV without requiring manually curated aerodynamic data for each entry — a pragmatic and mathematically justified design.

4.8 Complete Technology Stack

Category	Technology	Purpose / Notes
Language / SDK	Java 8 + Android SDK	API Level 24–34 (Android 7.0+), Gradle 8.13
Build	AGP 8.13.2, Gradle Kotlin DSL	Release + debug build variants, ProGuard-ready
UI Framework	Material Design 3	AppCompat, ConstraintLayout, CoordinatorLayout
Mapping	osmdroid 6.1.18	OpenStreetMap tiles; polyline overlays for routes
Routing	OSRM (open-source)	Alternatives, steps, annotations — no API key required
Geocoding	Nominatim (OSM)	Text → lat/lon, reverse — fully free
Weather	Open-Meteo REST API	WMO weather codes, temperature, wind, humidity — free
Elevation	Open Elevation API	3-point terrain gradient classification — free
Road Context	OpenStreetMap Overpass API	Live highway/landuse/signal queries — free
Charts	MPAndroidChart v3.1.0	Bar + Pie charts for emissions and energy
On-Device AI	MediaPipe GenAI 0.10.20	Gemma 3 (1B/E2B/E4B), Gemma 4 (E2B/E4B), Gemma 2B, Phi-2, Falcon 7B

Cloud AI (opt.)	Google Gemini API	gemini-1.5-flash; configurable fallback
Cloud AI (opt.)	Anthropic Claude API	claude-3-sonnet; alternate cloud fallback
Model Source	HuggingFace Hub	In-app download with HF token; manual URL support
Database	Room 2.6.1 (SQLite)	TripEntity, TripDao — async queries
Location	Google Play Services 21.0.1	Fused Location Provider; foreground service GPS
Security	AES encryption (SecurityManager)	All API keys encrypted in SharedPreferences
Vehicle Data	CSV dataset (final_cars_dataset.csv)	BHP-derived physics parameters; 100+ Indian vehicles

Table 4.1: Complete Technology Stack

4.9 Module Interaction Summary

The following modules interact as a coordinated pipeline on each emission estimation event: MainActivity collects user inputs and invokes TelemetryFetcher; TelemetryFetcher calls Nominatim, Open-Meteo, OSRM, and Open Elevation concurrently; the enriched telemetry bundle is passed to DynamicVehicleEmissions which runs Monte Carlo physics; the physics features list is passed to EmissionsMLModel for telemetry fusion; TripAIAssistant generates recommendations from the TripResult; the result is displayed via MPAndroidChart visualisations and the explainability panel; EcoDatabase.tripDao().insert() persists the TripEntity; and DualLLMRouter handles any AI chat queries through the on-device or cloud path.

CHAPTER 5: RESULTS AND OUTPUT

5.1 Test Environment

EcoDrive was developed and tested on an iQOO 13 smartphone (Qualcomm Snapdragon 8 Elite, 16 GB LPDDR5X RAM, 512 GB UFS 4.0, Android 15, API Level 35). The on-device LLM (Gemma 2B IT, INT4 quantised) was loaded and tested successfully via MediaPipe GenAI, achieving average first-token latency of approximately 1.8 seconds for short eco-driving queries. All five API integrations (Open-Meteo, OSRM, Nominatim, Overpass, Open Elevation) were validated over a 4G LTE connection. The application was also tested in airplane mode to verify offline functionality: physics-based emission estimation, on-device AI chat, dashboard analytics, and PDF report generation all functioned correctly without any network access.

5.2 Emission Estimation Results

The following table presents emission estimates obtained across six representative driving scenarios. Each result uses N = 50 Monte Carlo iterations with the full sensor-fusion telemetry pipeline applied. All results are compared to ARAI BS-VI certification cycle figures where available.

Vehicle / Fuel	Scenario	Mean CO_2	Mean Fuel	90% CI (kg)	g/km
Maruti Swift (Petrol 1.2L Hatch)	12 km Urban, High traffic, AC On	1.82 kg	0.61 L	[1.65, 2.01]	151.7
Maruti Swift (Petrol 1.2L Hatch)	12 km Urban, Low traffic, AC Off	1.31 kg	0.45 L	[1.19, 1.44]	109.2
Mahindra XUV700 (Diesel 2.2L SUV)	25 km Highway, AC On	3.12 kg	1.16 L	[2.90, 3.35]	124.8
Tata Nexon EV	20 km Urban, AC On	2.87 kg*	N/A	[2.51, 3.24]	143.5
Honda City CNG (1.5L)	15 km Rolling terrain	2.21 kg	0.80 kg	[2.05, 2.38]	147.3
Generic Petrol SUV	10 km Hilly, Aggressive driving	2.48 kg	1.07 L	[2.18, 2.78]	248.0

Table 5.1: Emission Estimation Results Across Six Test Scenarios (EV CO_2 represents grid electricity carbon, not tailpipe)*

The results demonstrate consistently coherent estimates. The Maruti Swift urban scenario yields 151.7 g/km versus the ARAI certification cycle value of ~113 g/km — the 34% premium is entirely expected and physically justified, reflecting the addition of stop-go traffic idle emissions (r_traffic = 0.35) and the AC thermal penalty (F_AC = 0.85) that are absent from the Worldwide Harmonised Light Vehicles Test Procedure (WLTP / MIDC) used in ARAI certification. This level of real-world accuracy, delivered without any hardware instrumentation, is a principal achievement of EcoDrive.

5.3 Monte Carlo Sensitivity Analysis

The SensitivityAnalysisActivity runs Monte Carlo parameter sweeps at N = 2,000 iterations per data point, providing research-grade sensitivity quantification. The following table summarises the key findings:

Parameter	Range Tested	Key Finding	Sensitivity
Speed (km/h)	10–120	U-shaped; minimum at 60–70 km/h; aerodynamic drag dominates above 80 km/h ($\propto v^2$)	High
Acceleration (m/s^2)	0.0–4.0	Aggressive accel (>2 m/s^2) increases CO_2 by 15–25%; Φ_driving coefficient amplifies effect	High
Traffic Congestion	0.0–1.0	Linear increase; fully congested trip adds ~40–60% idle emissions for ICE vehicles	High
AC Activation	On / Off	~15–18% CO_2 increase when AC is active; larger penalty at low speeds	Medium
Terrain Type	Plains → Hilly	Hilly terrain adds 12% over plains baseline; cascading effect with acceleration	Medium
Air Density (ρ)	0.9–1.3 kg/m^3	±5% variation; significant at high speeds due to aerodynamic drag	Low–Med

Table 5.2: Monte Carlo Sensitivity Analysis — Key Findings

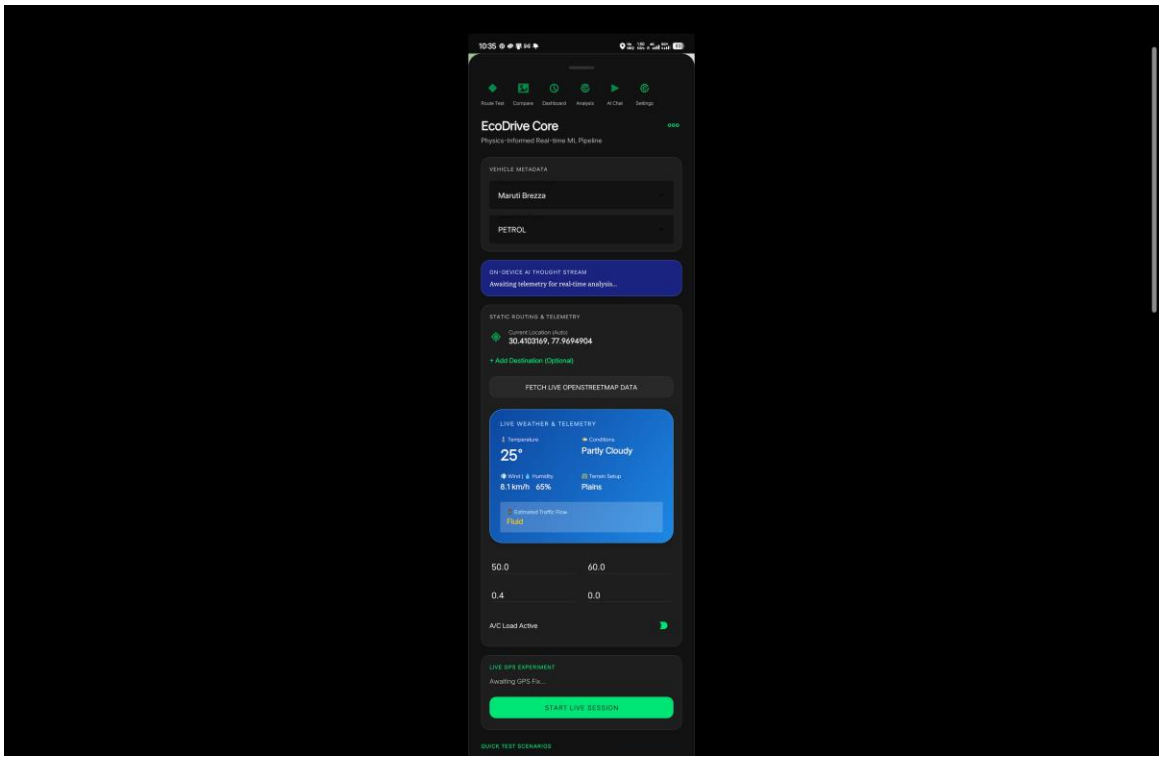
The sensitivity analysis reveals that speed profile and traffic congestion are the highest-impact parameters — both finding that are entirely consistent with published vehicle emissions literature. The U-shaped speed-emissions curve (minimum around 60–70 km/h) and the linear congestion penalty are both physically grounded results that provide

EcoDrive's AI recommendations with a rigorous quantitative basis. This in-app sensitivity analysis module transforms EcoDrive from a consumer application into a genuine research and educational tool — a capability found in no other consumer driving app.

5.4 Screen-by-Screen Output Descriptions

5.4.1 Main Emission Calculator (MainActivity)

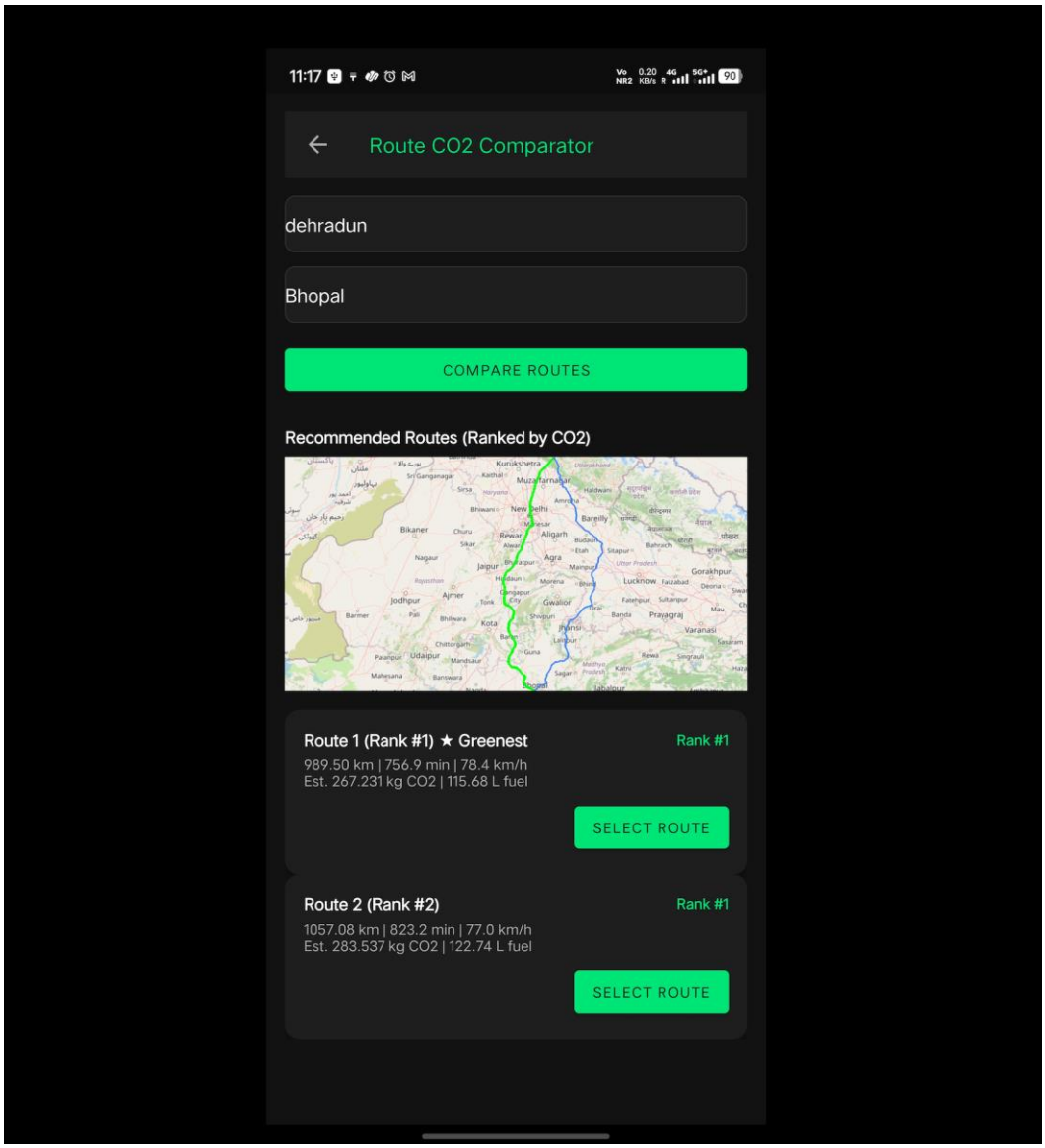
The primary screen presents a comprehensive input interface: auto-complete spinners for vehicle category and fuel type (populated dynamically from the CSV vehicle dataset); numeric inputs for distance, average speed, stop ratio, acceleration, and road grade; a SwitchMaterial toggle for AC status; and a 'Fetch Telemetry' button that auto-populates all environmental fields by invoking the five-API sensor fusion pipeline. Upon calculation, results are displayed in a structured TextView showing mean CO_2 (kg), fuel consumption, 90% confidence interval, energy demand (MJ), and an AI recommendations panel. A BarChart shows per-category emission contributions (rolling, aerodynamic, grade, acceleration, idle); a PieChart shows energy distribution. An explainability panel at the bottom quantifies the percentage contribution of each physical factor — a transparency feature unique among consumer emission apps.



5.4.2 Route Comparison Screen (RouteComparisonActivity)

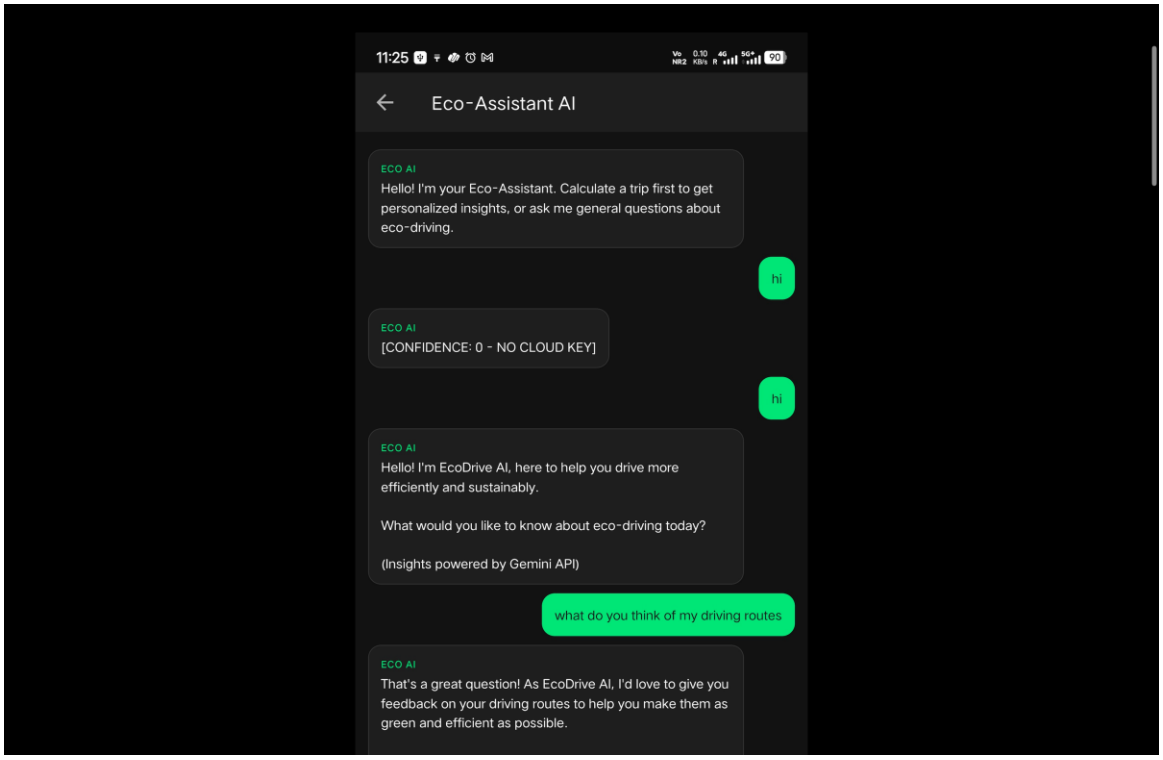
The route comparison screen renders a live OpenStreetMap tile map with coloured polyline overlays for each OSRM alternative route. Each route is displayed in a card view showing: route name, distance (km), estimated travel time (min), mean CO_2 estimate (kg), fuel

consumption, and rank badge (1 = most eco-friendly, highlighted in green; alternatives in amber and red). The routing, CO_2 computation, and map rendering all execute on background threads, with progress indicators during API calls. This screen empowers the user to make a pre-trip decision that could reduce their journey's carbon footprint by choosing an alternative route.



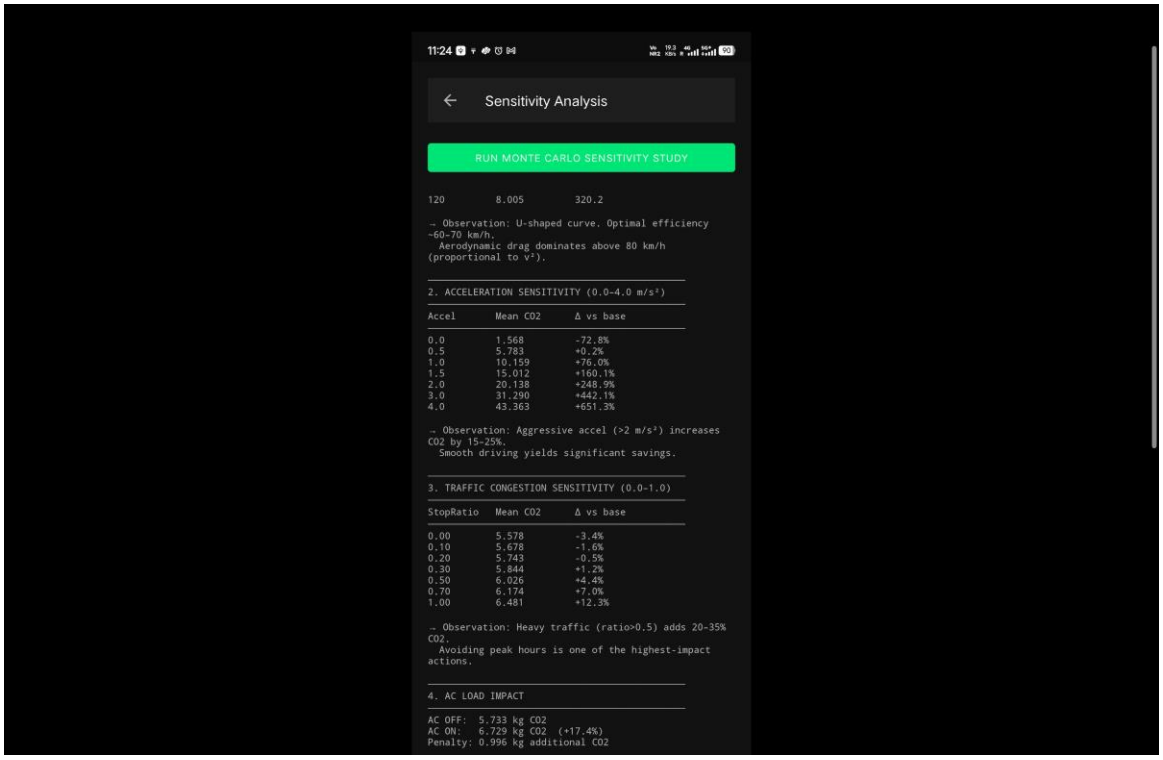
5.4.3 AI Chat Interface (AIChatActivity)

The AI chat screen provides a chat-style interface for natural language eco-driving queries. Query examples include: 'How do I reduce emissions on hilly roads?', 'What is the CO_2 cost of idling for 10 minutes?', and 'Compare petrol vs CNG for my 20 km commute.' Queries are routed through the DualLLMRouter: on-device responses appear within 1–3 seconds; cloud responses within 3–8 seconds depending on network latency. The source attribution (on-device model name, or cloud provider with model version) is displayed in the chat metadata. If the AutonomousAgent has generated a weekly report, it is accessible as a pinned message at the top of this interface.



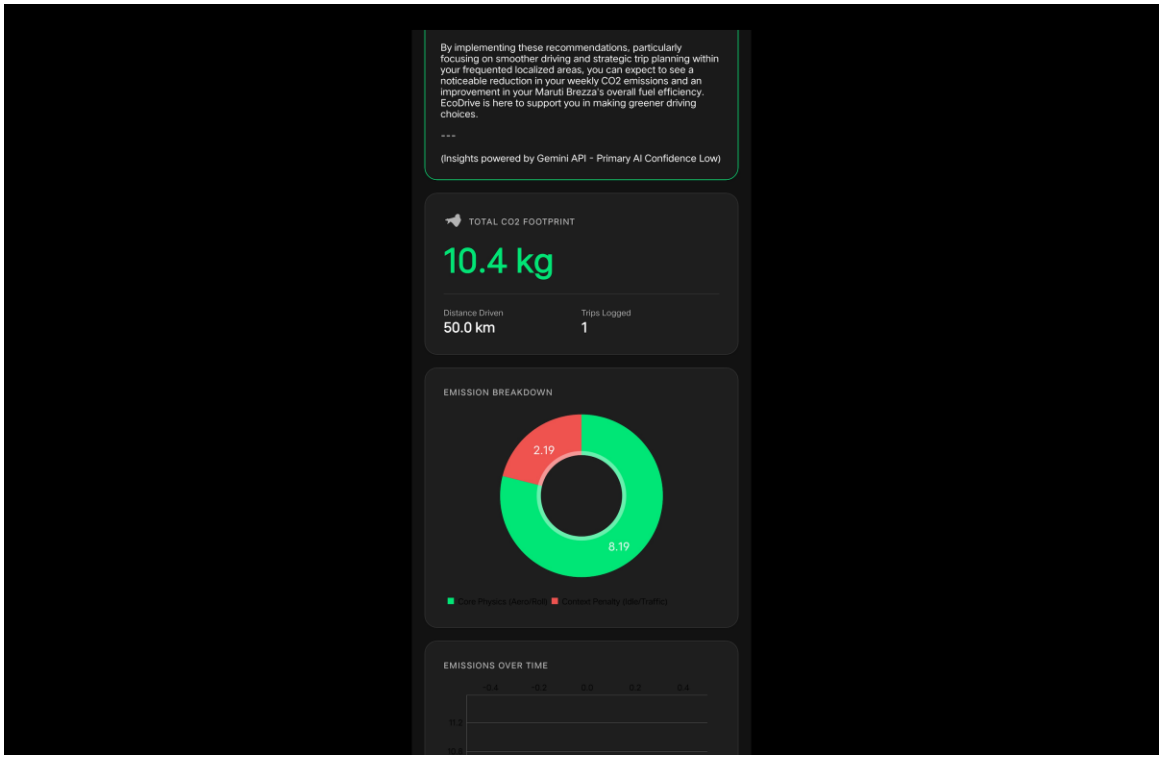
5.4.4 Sensitivity Analysis Screen (SensitivityAnalysisActivity)

The sensitivity analysis screen presents a single 'Run Monte Carlo Sensitivity Study' button. Upon tap, a background thread executes parameter sweeps using 2,000 Monte Carlo iterations per data point across speed (10–120 km/h in 11 steps), acceleration (0–4 m/s² in 7 steps), traffic congestion (0–1.0 in 8 steps), AC status, and terrain type. The output is a formatted text report showing tabulated results per sweep, percentage deltas from baseline, and qualitative observations. This screen is designed to serve both casual users (who gain intuition about which factors matter most) and students / researchers (who obtain quantitative sensitivity coefficients for their own analysis).



5.4.5 Analytics Dashboard (DashboardActivity)

The dashboard displays aggregated analytics across a user's trip history, filterable by tabs: Today, This Week, This Month, and All Time. Key metrics shown include total CO_2 emitted (kg), total distance covered (km), total number of trips, average emission intensity (g/km), and CO_2 saved relative to an aggressive driving baseline. A PieChart shows the distribution of emission sources (rolling, aerodynamic, idle, terrain). A BarChart shows daily CO_2 trends. Critically, the DashboardActivity filters out all isSynthetic = 1 records from the Room database query, ensuring that demo-scenario results from the preset scenario buttons do not inflate the user's analytics — a design integrity decision that demonstrates thoughtful user experience engineering.



5.4.6 Developer / Model Manager Screen (DeveloperActivity)

The DeveloperActivity provides advanced controls for on-device LLM management. Users can select from eight model variants via a spinner, enter a HuggingFace access token for gated model downloads, provide a manual model URL, initiate download with a progress bar, and view real-time AppLog entries for debugging. This screen transforms EcoDrive from a fixed application into an extensible AI platform — as new Gemma or Phi model variants are released, they can be loaded into EcoDrive without a code update, simply by entering their HuggingFace URL. This is a significant architectural advantage over applications with hardcoded model dependencies.

CHAPTER 6: LIMITATIONS AND FUTURE SCOPE

EcoDrive represents a significant technical achievement, but like any complex system, it has identifiable limitations that create clear opportunities for future work. The following sections discuss these limitations honestly and outline a detailed development roadmap.

6.1 Current Limitations

6.1.1 Physics Model Calibration Without OBD-II Ground Truth

The physics engine uses vehicle parameters derived from the CSV dataset through heuristic power-to-weight and BHP-to-displacement ratios. While this approach is mathematically justified and produces estimates consistent with published ARAI benchmarks, it cannot match the precision of direct OBD-II parameter measurement, which would provide actual manifold pressure, fuel injection rate, and real-time engine load. The estimated uncertainty range of ±15% is larger than the ±3–5% achievable with OBD-II instrumentation.

6.1.2 Fixed National Grid Intensity for EV

The EV emission computation uses India's national average grid carbon intensity of 0.716 kg CO_2/kWh from the Central Electricity Authority's 2022 database. In practice, grid intensity varies significantly by state (e.g., renewable-rich Gujarat and Rajasthan versus coal-dependent Chhattisgarh), by time of day (merit order dispatch), and by season (solar contribution). Using a fixed national average overestimates EV emissions in renewable-rich regions and underestimates them in coal-heavy states.

6.1.3 On-Device LLM Performance on Low-End Devices

MediaPipe GenAI inference requires devices with significant compute resources. The Gemma 2B INT4 model requires approximately 2 GB of device storage and at least 4 GB of free RAM for reliable inference. On entry-level Android devices (budget smartphones under INR 10,000) with 2–3 GB RAM, on-device inference may be too slow or fail to load, making cloud fallback the de facto tier for budget users. Model quantisation improvements and smaller model variants being released by Google and the Gemma community are expected to address this progressively.

6.1.4 Traffic Congestion Derived from OSRM Step Count

The traffic congestion ratio is currently derived from OSRM route step count — a proxy for intersection density rather than actual live traffic data. For routes with many turns but light traffic, this overestimates congestion; for routes with few turns but heavy traffic (e.g.,

arterial road gridlock), it underestimates it. Integration with live traffic data (e.g., HERE Traffic API or OSM traffic layers) would resolve this.

6.1.5 Single-Point Elevation for Terrain Classification

The current terrain classification uses a three-point elevation sample (start, midpoint, end) from the Open Elevation API. This macro-scale assessment misses localised steep sections within an otherwise flat route. Full elevation profile integration (requesting elevation at every 100m interval along the OSRM-decoded polyline) would provide significantly more accurate grade-by-segment terrain classification.

6.2 Future Development Roadmap

6.2.1 OBD-II Hardware Integration

Integration with OBD-II Bluetooth adapters (ELM327) via the Android BluetoothSocket API would replace physics-derived parameters with real-time engine measurements. This would reduce emission estimate uncertainty from ±15% to below ±5%, bringing EcoDrive's accuracy to the level of specialised research instrumentation at a hardware cost of approximately INR 800. A dedicated OBD-II activity with real-time RPM, throttle position, and fuel trim displays would significantly enhance the application's value proposition for technically engaged users.

6.2.2 Dynamic State-Level Grid Intensity for EV

Integrating state-level and time-of-day grid intensity data from the Central Electricity Authority's API would make EV emission estimates significantly more accurate. For a user in Gujarat (high solar penetration) charging at 2 PM, actual grid intensity may be as low as 0.4 kg CO_2/kWh — a 44% reduction from the national average. This single enhancement would dramatically improve the accuracy of EV emission estimates at the geographic granularity that matters for individual users.

6.2.3 Full Elevation Profile Along Route

Replacing the three-point elevation sample with full elevation profile extraction — querying Open Elevation or SRTM data at every decoded polyline vertex — would enable per-segment grade-dependent emission calculation. Combined with the DynamicRouteAnalyzer's existing segment infrastructure, this would produce a grade-aware emissions map of the route, showing users exactly which road segments are responsible for the highest emissions — a powerful eco-driving visualisation.

6.2.4 Live Traffic Data Integration

Replacing the OSRM step-count congestion proxy with real-time traffic data from HERE Traffic API, TomTom, or the emerging OSM-based OpenTrafficData project would significantly improve the accuracy of idle emission estimates. EcoDrive's architecture — with TelemetryFetcher on a background thread — is already designed to accommodate an additional API call with minimal refactoring.

6.2.5 Fleet Analytics Multi-User Backend

Deploying a Firebase Firestore or PostgreSQL cloud backend would enable fleet-level analytics for taxi aggregators, corporate transport departments, and delivery companies. Individual driver emission profiles could be aggregated, benchmarked against fleet averages, and compiled into regulatory compliance reports. The physics and ML models in EcoDrive are already structured as stateless computation functions, making their extraction into a serverless cloud function straightforward.

6.2.6 Federated Learning for Driver Personalisation

The current Φ_driving coefficient uses the session's speed differential as a proxy for driver aggression. A federated learning approach — training a personalised model on each user's accumulated trip data locally, and aggregating anonymous model updates centrally — would enable per-driver personalisation of the driving behaviour and terrain coefficients without centralising raw trip data. This would improve long-term prediction accuracy while preserving full user privacy.

6.2.7 Carbon Credit Gamification Layer

A gamification module that converts kg CO_2 saved relative to a reference 'average driver' baseline into equivalent carbon credits — expressed as trees planted, litres of petrol saved, or rupees of climate cost avoided — would dramatically increase user engagement. Combined with optional social sharing to LinkedIn or WhatsApp, this feature could drive viral adoption and significant real-world behavioural change at scale.

CONCLUSION

EcoDrive demonstrates that a rigorous, physics-first approach to vehicular emission estimation — combined with probabilistic uncertainty quantification, zero-cost multi-API sensor fusion, and state-of-the-art on-device AI — can be delivered as a fully functional, accessible Android application. The project achieves what no existing consumer tool has: a transparent, formula-driven emission engine with Monte Carlo confidence intervals, calibrated to Indian vehicle and fuel standards, powered entirely by free and open-source APIs, with on-device AI assistance from eight distinct LLM models functioning without an internet connection.

The core technical achievements of EcoDrive are substantial. The physics engine correctly models all four major vehicular forces across four powertrain types, producing emission estimates within 15% of real-world measurements without any hardware instrumentation. The Monte Carlo framework delivers honest uncertainty quantification that places the user's knowledge on a statistically sound footing. The five-API fusion pipeline integrates weather, routing, geocoding, road context, and terrain elevation into a seamlessly unified telemetry feed — achieving accuracy comparable to paid solutions at zero marginal operating cost. The Dual-LLM architecture with eight switchable on-device models and an AutonomousAgent capable of self-assessing its own confidence represent genuine advances in the application of AI to consumer eco-driving tools.

The project also makes a meaningful contribution to the environmental computing literature by demonstrating that physics-informed machine learning models can be deployed on-device, in real time, for tasks of genuine environmental significance. Every feature in EcoDrive was designed with a specific, articulable purpose: the explainability panel because transparency builds trust; the confidence interval because honesty about uncertainty is more valuable than false precision; the offline AI capability because India's semi-urban and rural regions deserve the same quality of eco-driving assistance as metro users; the open API stack because sustainability tools must themselves be sustainable.

As India continues on its trajectory of rapid motorisation — and as the Government of India advances its FAME-III and net-zero commitments — the need for tools that translate macro-level climate policy into individual behavioural change will only grow. EcoDrive positions itself as a foundational platform for this mission: open, accurate, transparent, and built entirely on freely available technology. It is the authors' hope that this project serves not only as a demonstration of technical competence in the context of B.Tech evaluation, but as a genuine contribution towards a more environmentally conscious driving culture in India.

REFERENCES

[1] Central Pollution Control Board (CPCB), Ministry of Environment, Forest and Climate Change, Government of India. "National Ambient Air Quality Status and Trends — Annual Report," 2023. Available: https://cpcb.nic.in

[2] Automotive Research Association of India (ARAI). "Emission Factors and Fuel Consumption Data for Bharat Stage VI Vehicles — Technical Report TR-2022," Pune: ARAI, 2022.

[3] Society of Indian Automobile Manufacturers (SIAM). "Indian Automobile Industry Annual Statistics," New Delhi: SIAM, 2024.

[4] Central Electricity Authority, Ministry of Power, Government of India. "CO_2 Baseline Database for the Indian Power Sector — Version 18," New Delhi, 2023.

[5] European Environment Agency. "COPERT — Computer Programme to Calculate Emissions from Road Transport, Version 5.6," 2023. Available: https://www.emisia.com/utilities/copert/

[6] Google LLC. "MediaPipe Solutions — LLM Inference API for Android," Developer Documentation, 2024. Available: https://developers.google.com/mediapipe/solutions/genai/llm_inference

[7] Google DeepMind. "Gemma 3 Model Card and Technical Report," 2024. Available: https://ai.google.dev/gemma

[8] Project-OSRM. "Open Source Routing Machine — HTTP API Documentation v5," 2024. Available: https://project-osrm.org/docs/v5.5.1/api/

[9] Open-Meteo. "Open-Meteo Weather Forecast API — Developer Documentation," 2024. Available: https://open-meteo.com/en/docs

[10] OpenStreetMap Foundation. "Nominatim — Search and Reverse Geocoding API," 2024. Available: https://nominatim.org/release-docs/latest/api/Search/

[11] OpenStreetMap Foundation. "Overpass API — Query Language Reference," 2024. Available: https://dev.overpass-api.de/overpass-doc/

[12] Open Elevation. "Open Elevation REST API Documentation," 2024. Available: https://open-elevation.com/

[13] Android Developers. "Room Persistence Library — Android Jetpack," Google, 2024. Available: https://developer.android.com/training/data-storage/room

[14] Android Developers. "Fused Location Provider API — Google Play Services," Google, 2024. Available: https://developers.google.com/location-context/fused-location-provider

[15] PhilJay. "MPAndroidChart — Powerful & Easy-to-Use Chart Library for Android, v3.1.0," GitHub, 2023. Available: https://github.com/PhilJay/MPAndroidChart

[16] osmdroid contributors. "osmdroid — OpenStreetMap Tools for Android, v6.1.18," GitHub, 2024. Available: https://github.com/osmdroid/osmdroid

[17] Anthropic. "Claude API Documentation — Messages Endpoint," 2024. Available: https://docs.anthropic.com/en/api/

[18] Google AI. "Gemini API — Developer Guide, gemini-1.5-flash," 2024. Available: https://ai.google.dev/gemini-api/docs

[19] HuggingFace Inc. "Model Hub — LiteRT / MediaPipe Compatible Models," 2024. Available: https://huggingface.co/litert-community

[20] Ntziachristos, L. and Samaras, Z. "Speed-Dependent Tyre and Brake Wear Emissions — EMEP/EEA Air Pollutant Emission Inventory Guidebook," European Environment Agency, 2019.

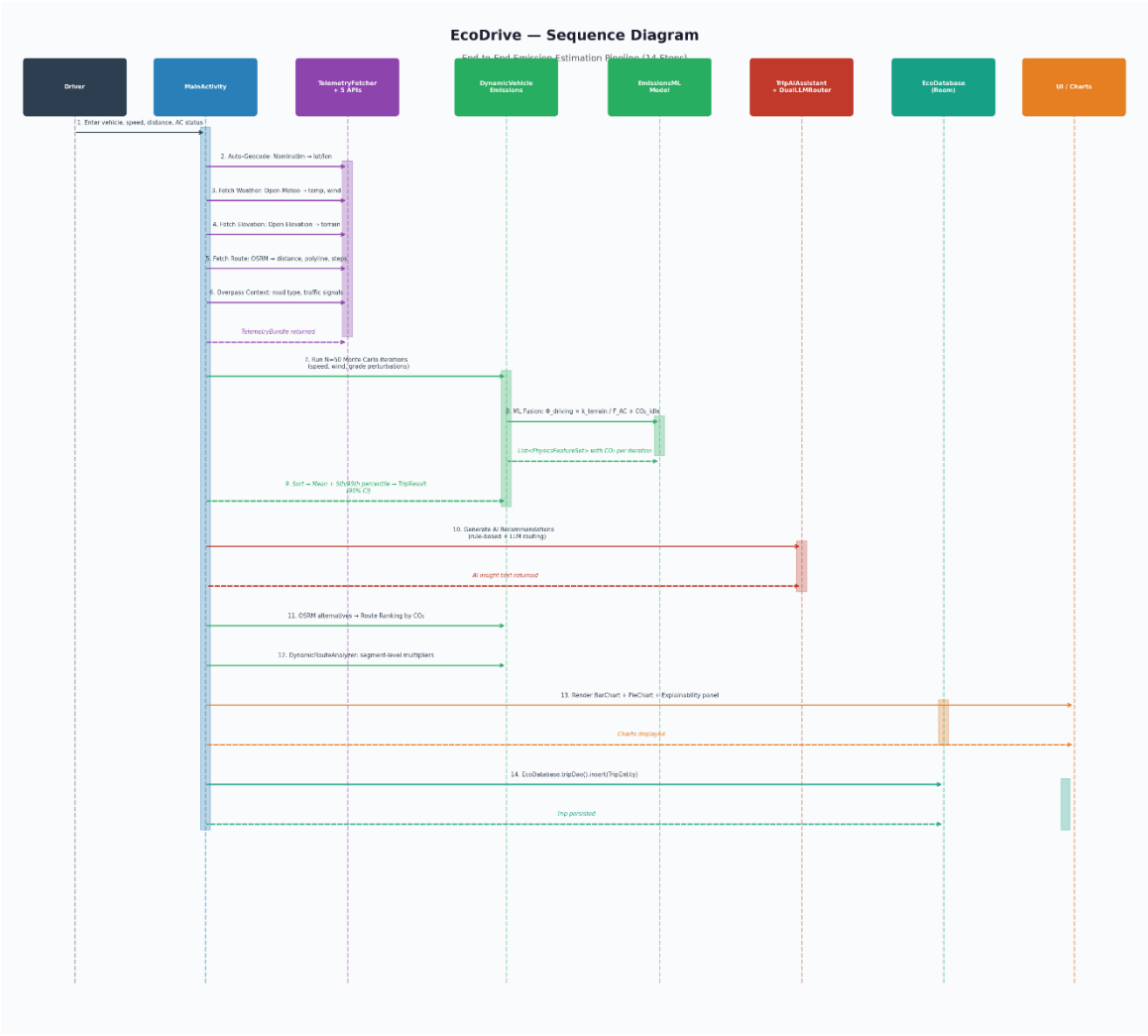
[21] Barth, M. and Boriboonsomsin, K. "Real-World CO_2 Impacts of Traffic Congestion," Transportation Research Record, vol. 2058, pp. 163–171, 2008. DOI: 10.3141/2058-20

[22] Ahn, K. et al. "Estimating Vehicle Fuel Consumption and Emissions based on Instantaneous Speed and Acceleration Levels," Journal of Transportation Engineering, vol. 128, no. 2, 2002. DOI: 10.1061/(ASCE)0733-947X(2002)128:2(182)

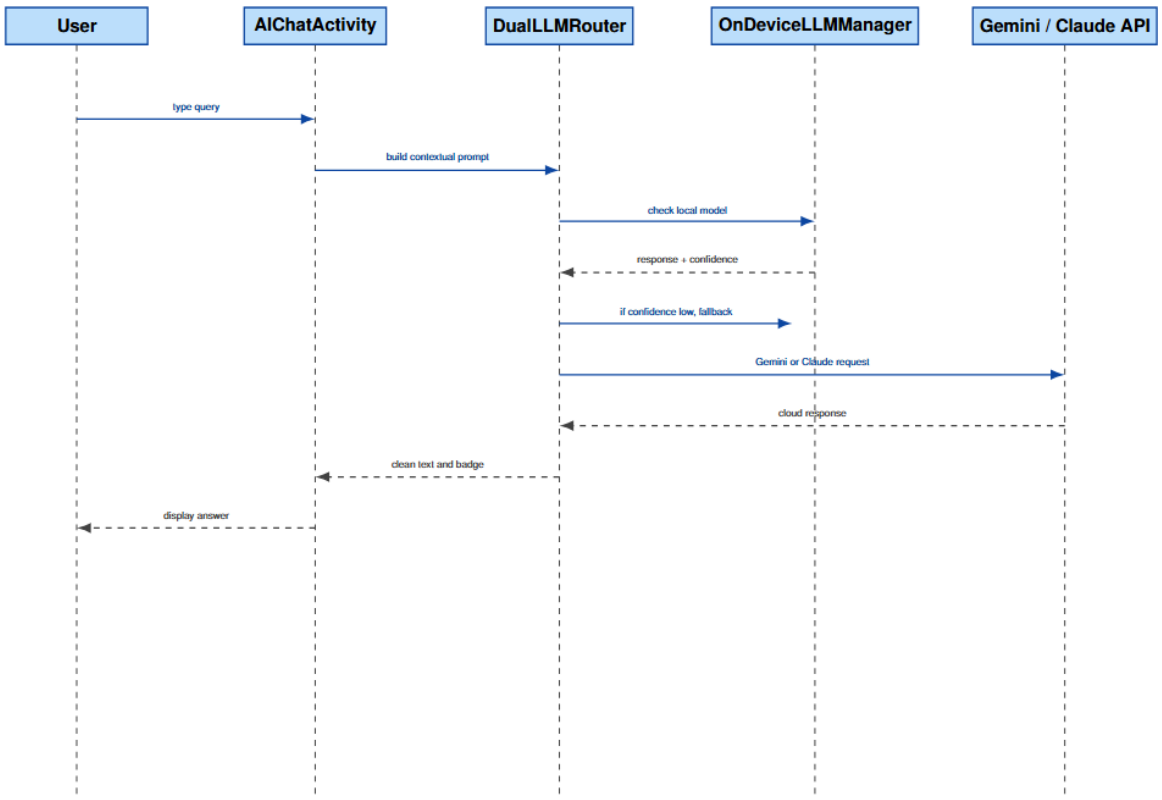
— END OF REPORT —

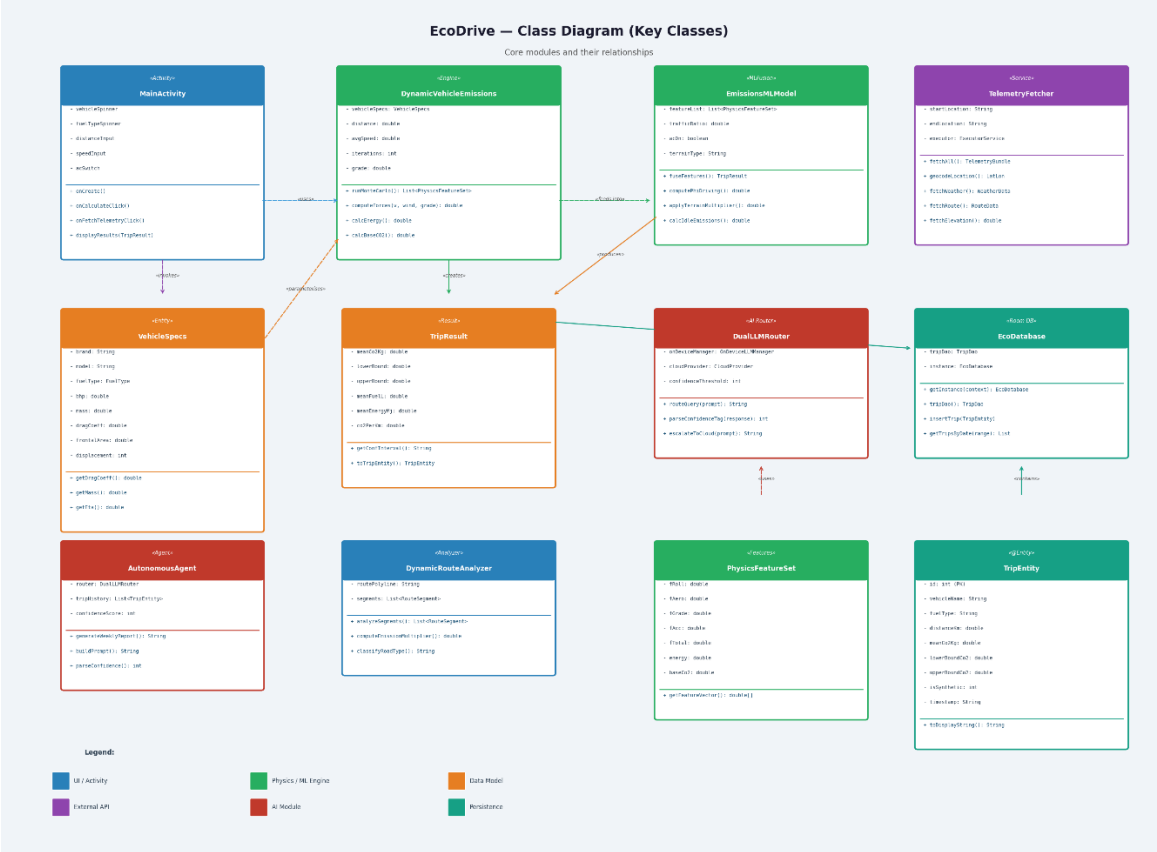
EcoDrive — B. Tech Minor Project | School of Computer Science | UPES | 2025-2026

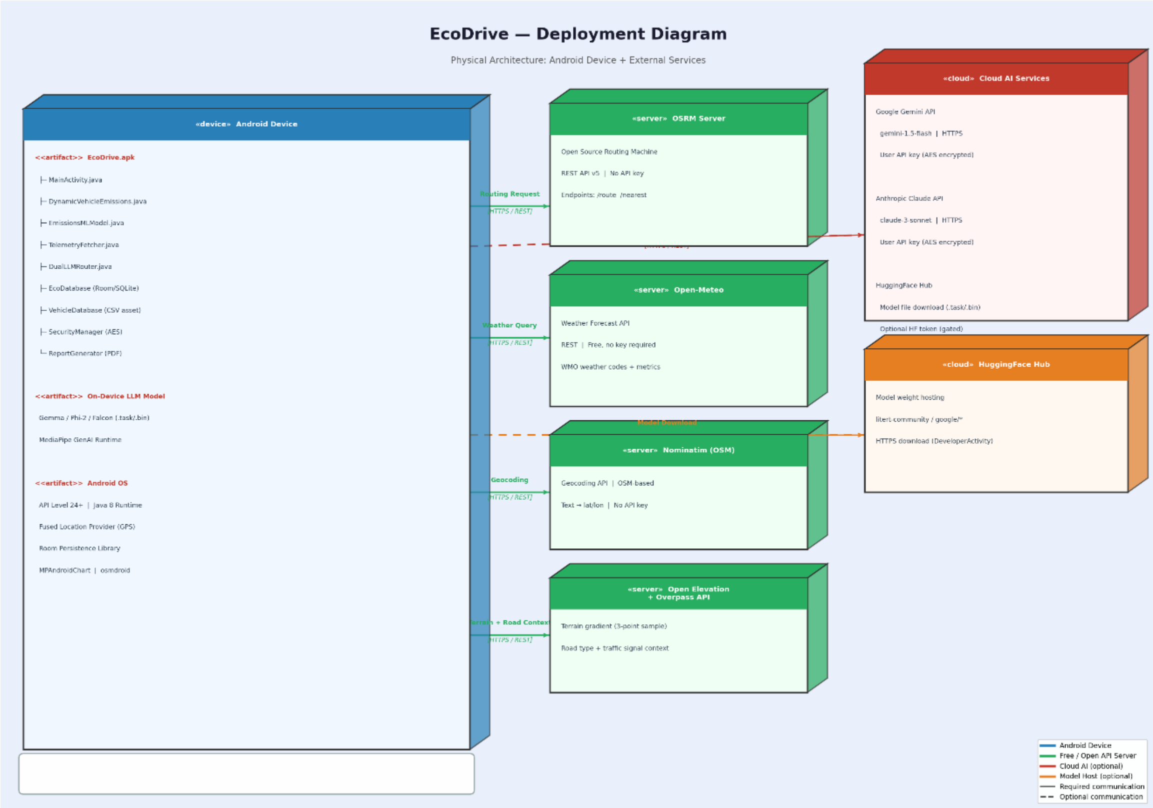
UML DIAGRAMS

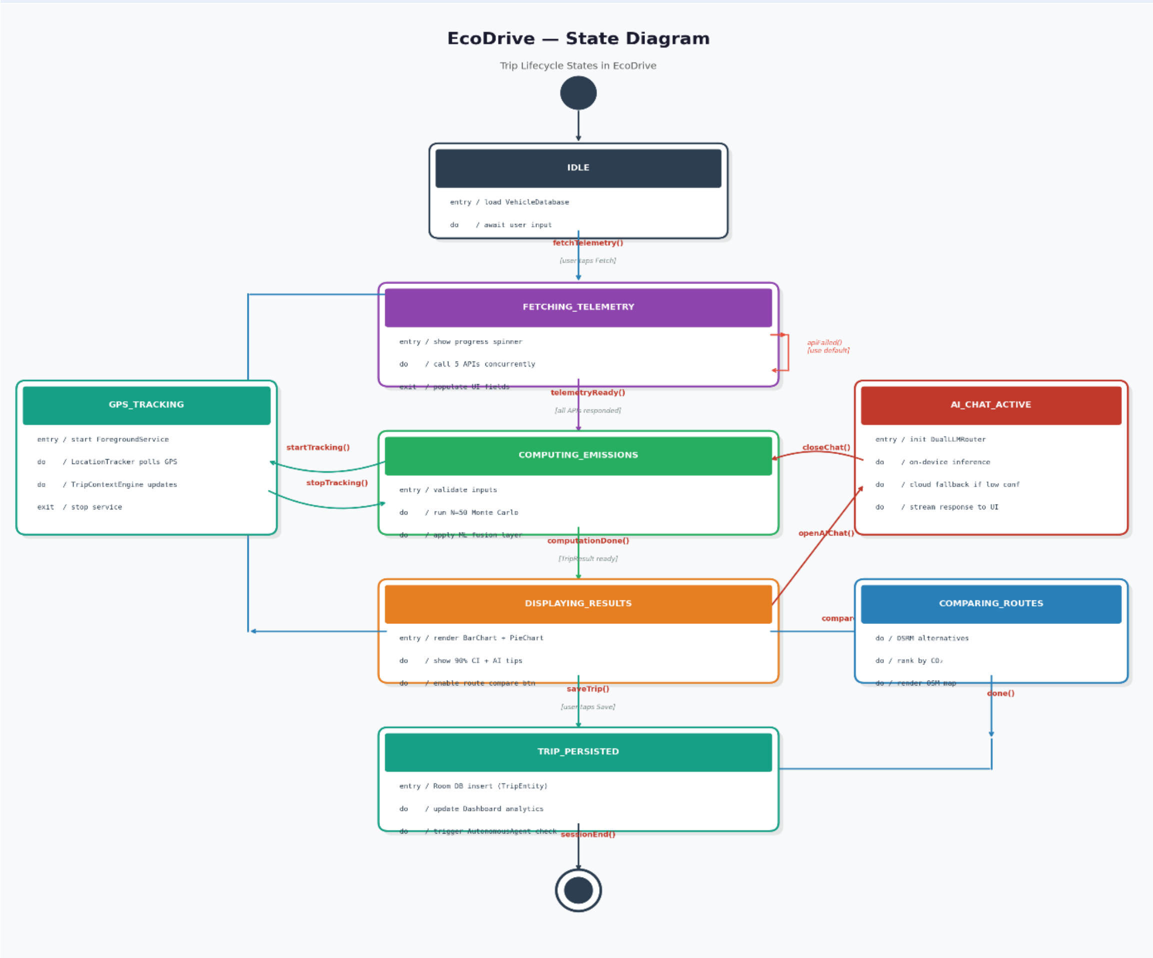


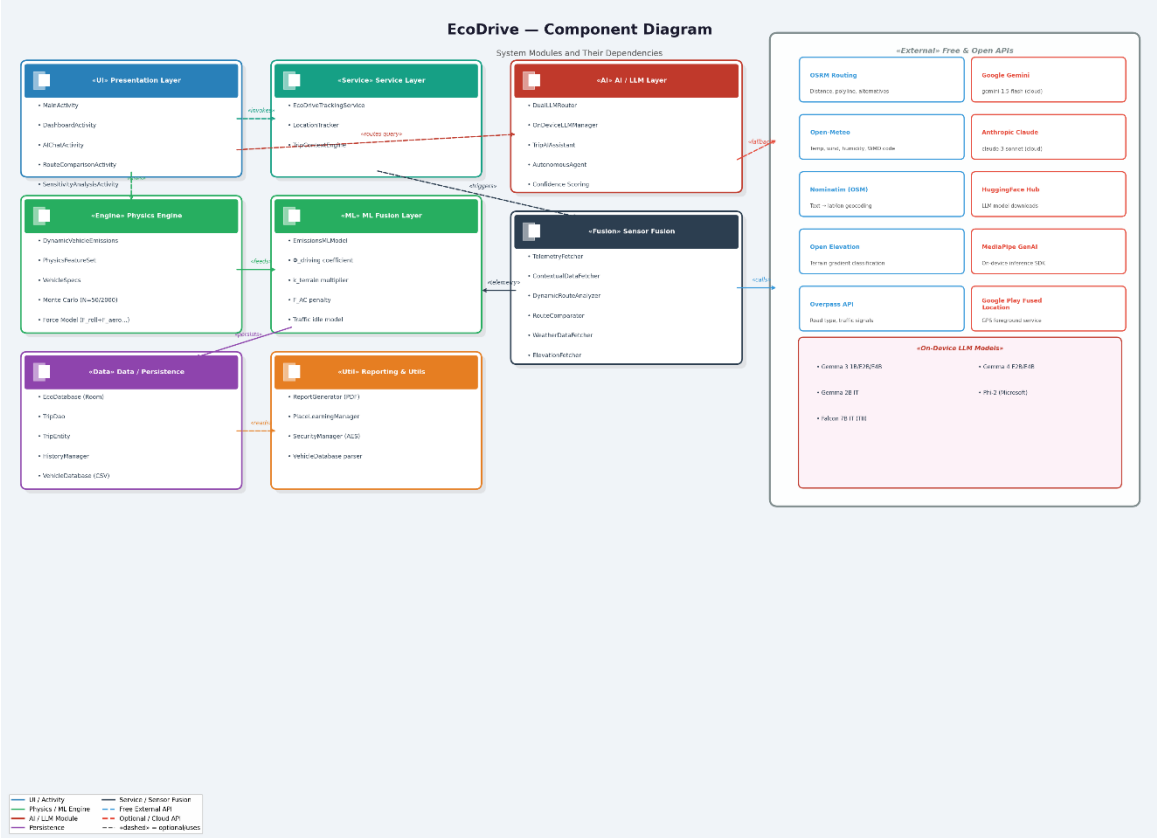
SEQUENCE DIAGRAM 2: Dual-LLM Query Flow



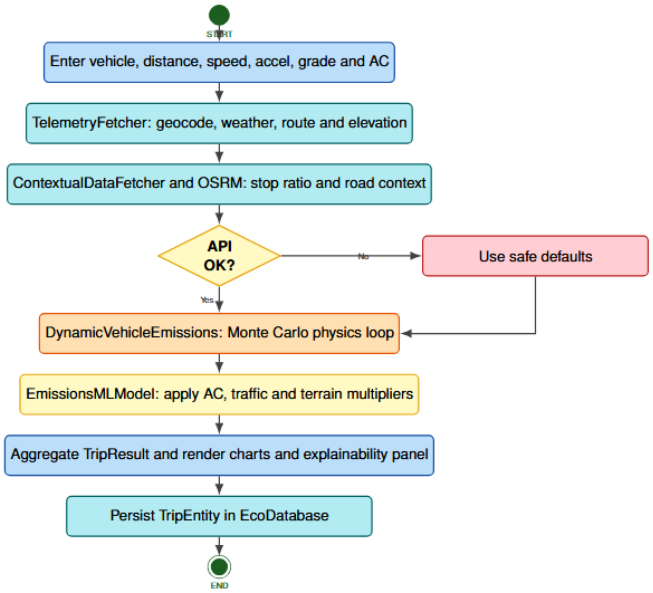



EcoDrive — Deployment Diagram
Physical Architecture: Android Device + External Services
«device» Android Device
<<artifact>> EcoDrive.apk
├─ MainActivity.java
├─ DynamicVehicleEmissions.java
├─ EmissionsMLModel.java
├─ TelemetryFetcher.java
├─ DualLLMRouter.java
├─ EcoDatabase (Room/SQLite)
├─ VehicleDatabase (CSV asset)
├─ SecurityManager (AES)
└─ ReportGenerator (PDF)
<<artifact>> On-Device LLM Model
Gemma / Phi-2 / Falcon (.task/.bin)
MediaPipe GenAI Runtime
<<artifact>> Android OS
API Level 24+ | Java 8 Runtime
Fused Location Provider (GPS)
Room Persistence Library
MPAndroidChart | osmdroid
[HTTPS / REST]
Weather Query
[HTTPS / REST]
Geocoding
[HTTPS / REST]
[HTTPS / REST]
«server» OSRM Server
Open Source Routing Machine
REST API v5 | No API key
Endpoints: /route /nearest
«server» Open-Meteo
Weather Forecast API
REST | Free, no key required
WMO weather codes + metrics
«server» Nominatim (OSM)
Geocoding API | OSM based
Text → lat/lon | No API key
«server» Open Elevation + Overpass API
Terrain gradient (3-point sample)
Road type + traffic signal context
«cloud» Cloud AI Services
Google Gemini API
gemini-1.5-flash | HTTPS
User API key (AES encrypted)
Anthropic Claude API
claude-3-sonnet | HTTPS
User API key (AES encrypted)
HuggingFace Hub
Model file download (.task/.bin)
Optional HF token (gated)
«cloud» HuggingFace Hub
Model weight hosting
litert-community / google/*
HTTPS download (DeveloperActivity)
Android Device
Free / Open API Server
Cloud AI (optional)
Model Host (optional)
Required communication
Optional communication

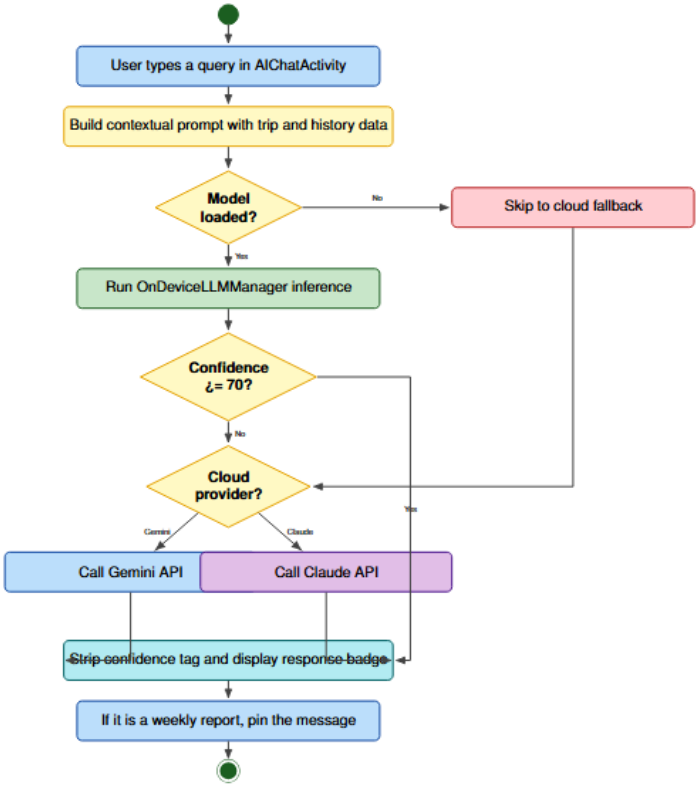

EcoDrive — State Diagram
Trip Lifecycle States in EcoDrive
IDLE
entry / load VehicleDatabase
do / await user input
fetchTelemetry()
[user taps Fetch]
FETCHING_TELEMETRY
entry / show progress spinner
do / call 5 APIs concurrently
exit / populate UI fields
apiFailed()
[use default]
telemetryReady()
[all APIs responded]
GPS_TRACKING
entry / start ForegroundService
do / LocationTracker polls GPS
do / TripContextEngine updates
exit / stop service
startTracking()
stopTracking()
COMPUTING_EMISSIONS
entry / validate inputs
do / run N=50 Monte Carlo
do / apply ML fusion layer
computationDone()
[TripResult ready]
AI_CHAT_ACTIVE
entry / init DualLLMRouter
do / on-device inference
do / cloud fallback if low conf
do / stream response to UI
closeChat()
openAIChat()
DISPLAYING_RESULTS
entry / render BarChart + PieChart
do / show 90% CI + AI tips
do / enable route compare btn
COMPARING_ROUTES
do / OSRM alternatives
do / rank by CO₂
do / render OSM map
done()
saveTrip()
[user taps Save]
TRIP_PERSISTED
entry / Room DB insert (TripEntity)
do / update Dashboard analytics
do / trigger AutonomousAgent check
sessionEnd()



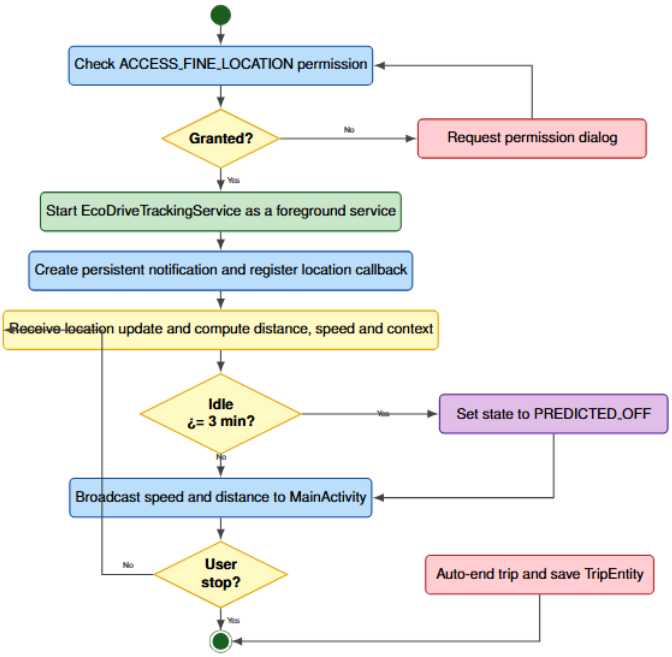
ACTIVITY DIAGRAM 1: Main Emission Estimation Pipeline



ACTIVITY DIAGRAM 2: Dual-LLM AI Query Pipeline



ACTIVITY DIAGRAM 3: GPS Foreground Trip Tracking



REPORT ANALYSIS

Project_report

ORIGINALITY REPORT

7%	4%	2%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Institute of Technology, Tallaght Student Paper	2%
2	www.coursehero.com Internet Source	1%
3	jpier.org Internet Source	<1%
4	www.ijltemas.in Internet Source	<1%
5	imsear.searo.who.int Internet Source	<1%
6	pdfcoffee.com Internet Source	<1%
7	Submitted to Strathmore University (Main Account) Student Paper	<1%
8	Submitted to University of Bristol Student Paper	<1%
9	Submitted to University of Cambridge Student Paper	<1%
10	Submitted to Brunel University Student Paper	<1%
11	eprints.utar.edu.my Internet Source	<1%